[University – to be confirmed]

Implementação de um sistema de gestão de conhecimento (KMS) que utilize machine learning para otimizar a resposta a incidentes de cybersegurança

[Author's full name – to be confirmed]

[Master's programme – to be confirmed]

[Submission month and year – to be confirmed]

[Department – to be confirmed]

Implementação de um sistema de gestão de conhecimento (KMS) que utilize machine learning para otimizar a resposta a incidentes de cybersegurança

[Author's full name – to be confirmed]

[Master's programme – to be confirmed]

Supervisors:
[Supervisor(s), qualifications and affiliation – to be confirmed]

[University – to be confirmed]
[Submission month and year – to be confirmed]

Acknowledgements

[Acknowledgements – to be confirmed]

Resumo

Durante a resposta a um incidente de cibersegurança, encontrar informação não basta: é necessário reconhecer a orientação aplicável, compreender as suas condições e decidir como a utilizar. Quando procedimentos, relatórios de incidentes e informação sobre ameaças se encontram dispersos, a experiência acumulada pode ser difícil de recuperar no momento em que é necessária. Neste trabalho, investigamos como um sistema de gestão de conhecimento pode utilizar *machine learning* para apoiar essa consulta e tornar o conhecimento disponível mais útil para a decisão.

Adotamos uma abordagem de investigação orientada à conceção, ligando a análise do problema à definição de requisitos, ao desenvolvimento de uma solução e à sua avaliação. Consideramos o ciclo de gestão do conhecimento, desde a recolha e organização até à atualização e reutilização. O papel da aprendizagem automática é estudado na recuperação de informação relevante e na formulação de orientações baseadas em fontes. A responsabilidade pela decisão mantém-se com o analista.

Definimos a otimização da resposta a incidentes através da qualidade da informação recuperada, da adequação das orientações e do esforço necessário para realizar as tarefas. A avaliação relaciona estas dimensões com a experiência de utilização, sem reduzir o desempenho a uma única medida de rapidez. Procuramos, assim, aproximar a gestão de conhecimento das necessidades da resposta a incidentes, preservando a possibilidade de consultar as fontes e respeitando a confidencialidade da informação.

Palavras-chave: gestão de conhecimento; aprendizagem automática; resposta a incidentes; cibersegurança; recuperação de informação.

Abstract

During a cybersecurity incident, finding information is only part of the task. An analyst must identify applicable guidance, understand its conditions and decide how to use it. When procedures, incident reports and threat information are dispersed, accumulated experience may be difficult to recover when it is needed. We investigate how a knowledge-management system can use machine learning to support this process and make documented knowledge more useful for incident-response decisions.

We adopt a design-science approach that connects the problem analysis with requirements, development and evaluation. Our scope covers the knowledge-management cycle, including collection, organisation, maintenance and reuse. We examine machine learning as a means of retrieving relevant information and formulating source-based guidance, while leaving responsibility for operational decisions with the analyst.

We define optimisation through the quality of retrieved information, the suitability of the guidance and the effort required to complete a task. The evaluation connects these dimensions with the experience of using the system, rather than reducing performance to a single speed measure. This perspective places knowledge management within the practical demands of incident response: making relevant experience accessible, retaining the means to examine its sources and respecting information confidentiality.

Keywords: knowledge management; machine learning; incident response; cybersecurity; information retrieval.

Contents

Acknowledgements i

Resumo iii

Abstract v

List of Figures xi

List of Tables xiii

List of Acronyms xv

1 Introduction 1
- 1.1 Motivation and context . . . 1
- 1.2 Research problem and approach . . . 2
- 1.3 Objectives and research questions . . . 2
- 1.4 Research methodology . . . 3
- 1.5 Scope and contribution . . . 4
- 1.6 Dissertation organisation . . . 4

2 State of the Art 7
- 2.1 Incident response and organisational knowledge . . . 7
- 2.2 The incident-response knowledge lifecycle . . . 8
- 2.3 Machine learning for retrieval and generation . . . 9
  - 2.3.1 Learned representations and model adaptation . . . 9
  - 2.3.2 Retrieval and source-grounded generation . . . 9
- 2.4 Related cybersecurity systems . . . 10
- 2.5 Reliability, confidentiality, and adversarial content . . . 12
- 2.6 Evaluating knowledge access and response decisions . . . 12
- 2.7 Synthesis and research gap . . . 13

3 Sentinel KMS: Design and Development 15
- 3.1 Design purpose and use cases . . . 15
- 3.2 Requirements and design priorities . . . 15
- 3.3 Architecture and allocation of responsibilities . . . 16
- 3.4 Knowledge organisation and maintenance . . . 18

3.4.1 Articles and curated incident playbooks . . . 18
3.4.2 Review, publication, and reuse . . . 18
3.4.3 Maintaining externally supplied knowledge . . . 19
3.5 Recoverable ingestion of local and remote sources . . . 19
3.6 Machine-learning retrieval and incident guidance . . . 20
3.6.1 Combining semantic and lexical retrieval . . . 20
3.6.2 Establishing the context for an answer . . . 21
3.6.3 Turning retrieved knowledge into response guidance . . . 21
3.6.4 Useful behaviour when generation is unavailable . . . 22
3.6.5 Keeping guidance attached to evidence . . . 22
3.7 Analyst interaction and knowledge reuse . . . 23
3.8 Confidentiality and operational choices . . . 26
3.9 Chapter synthesis . . . 26

4 Evaluation Design and Current Evidence 29
4.1 Purpose and evidential boundary . . . 29
4.2 Available artifacts and executed checks . . . 30
4.2.1 Datasets and evaluation mechanisms . . . 30
4.2.2 What the offline run establishes . . . 30
4.2.3 Executed software verification and scripted walkthrough . . . 31
4.3 Prospective corpus and comparison design . . . 32
4.4 Metrics and operational definitions . . . 33
4.4.1 Retrieval quality . . . 33
4.4.2 Answer quality and safety . . . 34
4.5 Expert review and adversarial validation . . . 35
4.6 Statistical and ethical protocol . . . 36
4.7 Requirement validation and limitations . . . 36

5 Conclusion 39
5.1 Conclusions supported by the present work . . . 39
5.2 Main contributions . . . 40
5.3 Limitations . . . 40
5.3.1 Literature and argument . . . 40
5.3.2 Implementation and deployment . . . 41
5.3.3 Evaluation and generalisation . . . 41
5.4 Future work and completion priorities . . . 41
5.5 Closing statement . . . 42

A Implementation Evidence and Reproducibility Boundaries 43
A.1 How the evidence should be read . . . 43
A.2 Worked design scenario: publication, citation, and withdrawal . . . 43
A.3 Evidence identity and verification limits . . . 44

B Source Evidence and Bibliographic Method **47**
B.1 Research inputs and limits of inspection 47
B.2 Claim-to-source examples 47

C Proposed Review and Analyst-Study Instruments **49**
C.1 Scenario selection and requirement coverage 49
C.2 Case review form 50
C.3 Answer review form 50
C.4 Analyst task record 51

References **53**

List of Figures

3.1 Sentinel's knowledge, application, and external-service boundaries. 17
3.2 A governed article's path from publication to access-checked evidence inspection. 23
3.3 Knowledge-library filters and problem-oriented playbook browsing (mocked data). 24
3.4 Ingestion stages and source-submission acknowledgement (mocked data). . 24
3.5 Assistant answer with article and exact-passage actions (mocked data). . . 25
3.6 Evidence dialog identifying the cited revision and passage (mocked data). . 25

List of Tables

2.1 Knowledge sources and evaluation focus in selected cybersecurity studies. . 11

3.1 Requirements guiding the design of Sentinel. 16

4.1 Observed engineering verification on the modified working tree, 29 September 2026; test counts are not participants or evaluated security questions. The report gate is a distinct synthetic-check layer, not part of the application-suite totals. 31

4.2 Requirement ID to implementation/test evidence and current status, linked to Chapter 3, Table 3.1. Passing tests establish selected assertions only; empirical acceptance remains pending. 38

A.1 Design-scenario checkpoints and their evidential status; these are not outcomes of an executed end-to-end case. 44

B.1 Illustrative claim-to-source boundaries in the literature argument. 47

C.1 Reader-facing scenario prompts mapped to the requirement identifiers in Table 4.2; no expert or analyst outcomes are reported. 49

List of Acronyms

AI	Artificial Intelligence
API	Application Programming Interface
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
CI	Continuous Integration
CTI	Cyber Threat Intelligence
CVE	Common Vulnerabilities and Exposures
DFIR	Digital Forensics and Incident Response
DSRM	Design Science Research Methodology
HNSW	Hierarchical Navigable Small World
HTTP	Hypertext Transfer Protocol
IOC	Indicator of Compromise
JDBC	Java Database Connectivity
JSON	JavaScript Object Notation
JWT	JSON Web Token
KEV	Known Exploited Vulnerabilities
KG	Knowledge Graph
KMS	Knowledge Management System
LLM	Large Language Model
ML	Machine Learning
MRR	Mean Reciprocal Rank
nDCG	Normalized Discounted Cumulative Gain
NLP	Natural Language Processing
NVD	National Vulnerability Database
PPI	Prediction-Powered Inference
QA	Question Answering
RAG	Retrieval-Augmented Generation
RGB	Retrieval-Augmented Generation Benchmark
RLS	Row-Level Security
RRF	Reciprocal-Rank Fusion
SLM	Small Language Model
SOC	Security Operations Centre
SQL	Structured Query Language
SSE	Server-Sent Events
SSRF	Server-Side Request Forgery

STIX	Structured Threat Information Expression
UI	User Interface
URI	Uniform Resource Identifier

CHAPTER 1

Introduction

Responding to a cybersecurity incident requires decisions about what has happened, which assets are affected and what action is appropriate. Analysts need to interpret the available evidence, relate it to operational conditions and select a defensible response. This dissertation investigates how knowledge management supported by machine learning can assist that work by making relevant knowledge easier to find, assess and reuse.

1.1. Motivation and context

Incident response involves detecting and analysing incidents, limiting their impact and restoring affected operations. The National Institute of Standards and Technology (NIST) places these activities within wider cybersecurity risk management, connecting response with preparation and continuous improvement. Its recommendations emphasise the use of incident experience to improve procedures and inform subsequent decisions.[12] This relationship between action and learning provides the starting point for the present research: knowledge produced through one investigation should remain useful when another incident requires a response.

Consider an analyst investigating a suspected account compromise. An external advisory may describe the threat, an internal procedure may specify the permitted response, and a previous incident report may explain why a similar action was ineffective in a particular environment. These sources answer different parts of the same question. To use them together, the analyst needs to establish their relevance, recognise their limitations and understand whether the circumstances of the earlier case still apply. Finding a document is only part of this task; interpreting it is also necessary.

We use *knowledge management* to mean the deliberate capture, organisation, maintenance and reuse of knowledge for this decision-making purpose. The knowledge of interest includes documented procedures, explanations of earlier decisions and lessons derived from incident experience. Capturing an outcome without its rationale can leave later readers unable to judge its applicability. Maintaining that context gives knowledge reuse a more specific purpose than accumulating documents.

Knowledge-sharing research considers both the factors affecting exchanges between individuals and the relationship between organisational processes and their intended outcomes.[8, 20] Research on cyber threat intelligence sharing likewise identifies organisational and trust-related obstacles that extend beyond technical compatibility.[1] These

perspectives motivate an approach that treats the contribution and maintenance of knowledge as part of the incident-response problem, alongside the ability to retrieve it.

1.2. Research problem and approach

The problem addressed is the gap between retaining cybersecurity information and making it usable when a response decision is needed. Relevant knowledge may be dispersed across documents written for different audiences, expressed in different terms or detached from the experience that explains it. A response procedure can therefore be available without its relevance being apparent. The research concerns how a knowledge-management system can help analysts connect an incident question to applicable knowledge and understand the basis for using it.

Knowledge maintenance is central to this problem. Advice may become inappropriate as systems, threats or organisational procedures change. NIST notes that the diversity and changing detail of incident response make a single static publication insufficient for maintaining operational guidance.[12] In this study, we therefore treat currency and context as conditions of useful knowledge reuse. We also include provenance, meaning the identifiable origin and supporting basis of the knowledge presented, and confidentiality, meaning that knowledge is available only to its authorised audience.

We investigate machine-learning models trained on language data as a means of relating an analyst's question to relevant material and producing explanations informed by that material. Research on retrieval-supported language generation provides a basis for combining learned language capabilities with external knowledge sources.[9] We adopt this as a research direction for making documented experience accessible in the context of an analyst's question. The question is whether this support helps people reach justified decisions with less effort, while retaining the information needed to assess the guidance.

This approach connects knowledge management with decision support. A useful explanation needs to preserve the conditions under which advice applies and distinguish supported recommendations from uncertainty. Analysts retain responsibility for assessing operational consequences and choosing an action. Reliability and confidentiality are consequently requirements for knowledge reuse, considered together with relevance and ease of use.

1.3. Objectives and research questions

The overall objective is to design, implement and evaluate a knowledge-management system (KMS) that uses machine learning to optimise knowledge reuse during cybersecurity incident response. We define *optimisation* in practical terms: reducing the time and effort needed to locate applicable knowledge and helping analysts make correct, well-supported decisions. This definition gives decision quality equal importance to speed.

The specific objectives are to identify the knowledge and maintenance practices needed

for incident-response tasks; to develop an approach that connects that knowledge with relevant, traceable guidance; and to evaluate its performance and usability within the conditions governing access to the knowledge. These objectives lead to the principal research question:

> How can a knowledge-management system use machine learning to support faster, better-informed incident-response decisions while preserving the reliability and confidentiality of the knowledge used?

Three research questions organise the investigation:

RQ1: What knowledge must be captured, organised and maintained to support incident-response decisions?

RQ2: How can machine-learning-based retrieval and language generation turn that knowledge into relevant, traceable guidance?

RQ3: How does the resulting system perform in retrieval, response quality, task efficiency and usability, subject to access and provenance controls?

Together, these questions connect the knowledge-management requirements to the proposed support for analysts and to its evaluation. They keep the research focused on the use of knowledge in a response task rather than on language generation in isolation.

1.4. Research methodology

We adopt Design Science Research Methodology (DSRM), which organises research around the creation and evaluation of an artefact intended to address a practical problem. Peffers et al. define six activities: problem identification and motivation, definition of solution objectives, design and development, demonstration, evaluation, and communication.[14] This approach suits the investigation because the KMS provides a means of examining how knowledge-management requirements and machine-learning support work together in incident-response tasks.

Problem identification began with a literature review of incident response and knowledge sharing. We then conducted semi-structured interviews with a research specialist to identify use cases, user roles and system requirements. This qualitative input informed the definition of solution objectives and the knowledge-management requirements examined under RQ1. Design and development address those objectives through the research artefact; Chapter 3 explains the design choices and their relationship to the problem.

Demonstration concerns how the approach applies to incident-response situations. Evaluation examines how well it meets the stated objectives.[14] We use this distinction to connect examples of knowledge reuse with a separate assessment of performance and usefulness. Communication brings the research argument, design rationale, methods and

findings together in this dissertation so that readers can assess the relationship between the problem and the conclusions.

The evaluation framework combines model-oriented and human-centred perspectives. Retrieval quality concerns whether the material found is relevant and sufficient for the question. Response quality concerns correctness, completeness and support for the explanation in its sources. Task efficiency concerns the time and effort required to reach a justified response decision, while usability concerns how readily people can find, interpret and assess the guidance. Access restrictions and provenance are conditions across these dimensions: relevant material must be appropriate for the intended reader, and the basis of advice must remain inspectable.

These dimensions are considered together when interpreting performance. Task completion time must be interpreted alongside the correctness and support of the resulting decision. Similarly, a response that reads clearly still requires assessment of its correctness and applicability. The evaluation therefore connects properties of retrieval and generated guidance with the analyst's use of that guidance, rather than treating either perspective as sufficient on its own. Chapter 4 presents the methods, results and interpretation.

1.5. Scope and contribution

The scope is knowledge-based decision support for cybersecurity incident response. It includes the use of procedures, threat-related documentation and recorded incident experience to inform an analyst's understanding of a situation. It also includes the capture and maintenance needed to make that material reusable. The intended users are analysts consulting knowledge and the contributors responsible for keeping it relevant.

The study focuses on assistance with interpretation and response planning. Autonomous remediation and the replacement of professional judgement fall outside its scope. The contribution is an approach linking incident-response knowledge requirements to machine-learning-supported guidance, together with an evaluation of that approach. Its interpretation is tied to the knowledge, tasks and participants considered in the study, providing a basis for discussing where the approach is useful and what limits its application.

1.6. Dissertation organisation

Chapter 2 reviews incident response, knowledge management and machine-learning approaches to retrieval and language generation, establishing the conceptual basis for the research. Chapter 3 presents the requirements, design and development of the proposed KMS and explains the rationale for its design choices.

Chapter 4 presents the evaluation methods and results, relating performance and human-centred assessment to the research questions. Chapter 5 draws together the answers, contributions, limitations and directions for further research. The appendices pro-

vide supporting research instruments and scenario and method details.

CHAPTER 2

State of the Art

An incident-response decision requires knowledge that applies to the affected environment, supports a defensible course of action, and remains available for examination. We review organisational knowledge sharing, machine-learning representations, retrieval-augmented generation, and evaluation from this perspective. The chapter offers a focused narrative synthesis rather than an exhaustive systematic review. Its comparison of cybersecurity studies considers the task, knowledge sources, model adaptation, and evaluation method, since improvements on different tasks cannot be interpreted as a common measure of incident-response effectiveness.

2.1. Incident response and organisational knowledge

NIST Special Publication 800-61r3 places incident response within continuing cybersecurity risk management. In its Cybersecurity Framework (CSF) 2.0 model, Detect, Respond, and Recover cover incident handling, while Govern, Identify, and Protect support preparation and risk reduction. Lessons learned inform improvements across these activities rather than belonging only to a final reporting stage. NIST also distinguishes an observable event from an incident: interpreting evidence is necessary to establish whether an incident has occurred.[12] This distinction matters for knowledge support. Retrieving a description of an attack does not establish that the attack explains an observed event.

Consider an analyst investigating suspected account compromise. A threat report can explain a technique, an internal procedure can specify an approved response, and a previous investigation can document exceptions encountered in a similar environment. The analyst still needs to assess which observations support the suspicion and whether the procedure applies. This example separates case evidence from reusable knowledge. Event records describe what was observed; response knowledge supplies the concepts, procedures, and prior experience needed to interpret those records. Their connection is a judgement about applicability, not simply a match between keywords.

We use *knowledge-management system* (KMS) to encompass the organisational practices and information services through which knowledge is captured, organised, maintained, shared, and reused. Ipe's conceptual framework locates knowledge sharing in interactions between individuals, while Yeboah distinguishes its enablers, processes, and outcomes.[8, 20] These perspectives explain why a searchable collection alone is insufficient: useful experience must first become accessible to others, and its reuse must serve an

organisational objective. For incident response, that objective is to help people reach correct, well-supported decisions with less effort. Faster production of an answer is valuable only when its interpretation and use remain appropriate.

2.2. The incident-response knowledge lifecycle

Knowledge capture concerns both formal guidance and experience gained during investigations. NIST recommends documented procedures, including playbooks that organise actionable tasks for particular situations, and the incorporation of lessons learned into improved practice.[12] A useful incident lesson records more than the final action: it explains the conditions under which the action was chosen and what evidence justified it. Otherwise, later responders may copy an outcome without understanding its constraints. The distinction between recording an incident and producing reusable guidance follows from the knowledge-sharing emphasis on creating, sharing, and applying knowledge.[8]

Organisation makes those conditions discoverable. Osman et al. compare knowledge-management tools through dimensions including knowledge sources, extraction, input, retrieval, and sharing, placing search within a broader process.[13] Applied to incident response, this perspective suggests describing knowledge by its purpose and context: affected technology, incident type, intended audience, author or responsible team, and date of applicability. A vendor advisory and an internal recovery procedure may concern the same vulnerability while answering different questions. Relevance therefore includes what a document enables the responder to do, not only whether it discusses the same topic.

Shared vocabularies can reduce ambiguity when knowledge crosses organisational boundaries. Takahashi and Kadobayashi's reference ontology coordinates cybersecurity operational information and existing information schemas.[19] An ontology defines concepts and their relationships; a knowledge graph represents particular entities and relations using such a vocabulary. These approaches can connect an attack technique to affected assets or mitigations, but structured representation does not establish that a relationship is correct. Li et al. identify noise and conflict during graph construction as sources of misleading inference.[10] For knowledge reuse, consistent terminology and reliable source support matter before more elaborate reasoning becomes useful.

Maintenance addresses changes in both the environment and the knowledge itself. Dhingra et al. demonstrate the difficulty of representing time-sensitive facts in language models trained on data snapshots.[4] The corresponding KMS problem is broader than model freshness: a formerly valid procedure can become unsuitable after a platform change, and an incident hypothesis can be corrected after further investigation. Current operational advice and a historical explanation may therefore require different versions of a source. Recording validity periods, review responsibility, and superseded guidance allows those uses to remain distinguishable rather than treating the newest text as appropriate for every question.

Sharing also depends on trust and permitted disclosure. Abraham et al.'s interview-

informed perspective on cyber threat intelligence (CTI) stresses human interpretation, incompatible platforms, and different meanings of trust.[1] A relevant internal lesson may contain information that cannot be shared with another team or organisation. Knowledge quality, topical relevance, and permission to use the material are separate considerations. The lifecycle closes when responders can report an incorrect or incomplete recommendation and responsible contributors can revise the underlying knowledge. This feedback connects retrieval to organisational learning instead of allowing errors to recur in later answers.

2.3. Machine learning for retrieval and generation

2.3.1. Learned representations and model adaptation

Machine learning can support knowledge access without requiring a new incident classifier. A classifier assigns an input to a defined category, such as malicious or benign, whereas a representation model produces features used to compare inputs. A large language model (LLM) used for generation produces text conditioned on its input. These roles can coexist, and the task differs from the method used to train the model. Sentence-BERT, for example, learns representations through fine-tuning but subsequently uses the resulting embeddings for similarity comparisons.[15]

An embedding is a numerical representation of text. Sentence-BERT adapts a pretrained language model so that sentences can be encoded separately and compared using measures such as cosine similarity. This permits document representations to be computed before a query arrives, avoiding a separate joint model evaluation for every possible sentence pair.[15] In an incident-response setting, semantic retrieval may connect a description of suspicious account activity with guidance phrased in different terminology. Similarity remains a retrieval signal, however: it does not establish that the guidance is current, authorised, or correct for the affected system.

Pretraining learns parameters from earlier data; inference uses those parameters to process new inputs. Using a pretrained embedding model is therefore an application of machine learning even when no local training occurs. Fine-tuning updates model parameters for a task or domain. Retrieval instead supplies selected external material at inference time, and few-shot prompting supplies examples in the input without itself changing the parameters. These mechanisms are compatible. Lewis et al.'s original retrieval-augmented generation approach combines pretrained retrieval and generation with task-specific fine-tuning; retrieval should not be defined as the absence of training.[9]

2.3.2. Retrieval and source-grounded generation

Lexical and semantic retrieval offer different routes to relevant knowledge. Exact terms matter for identifiers, product names, and error messages, while learned representations can recover material with related meaning. Hybrid retrieval combines these signals. A

reranker then reorders an initial candidate set using a further relevance assessment. RAG-Intel illustrates this sequence through keyword and dense retrieval followed by reranking and context reduction.[2] The attraction for incident response is practical: precise technical references and loosely described symptoms can occur in the same query. Whether combining retrieval methods improves a particular task remains an empirical question.

Retrieval-augmented generation (RAG) conditions a language model on external information selected for the current request. Lewis et al. distinguish knowledge encoded in model parameters from knowledge held in a retrievable collection. Their model combines a neural retriever with a sequence-to-sequence generator and evaluates the combination on knowledge-intensive language tasks.[9] This separation provides a route for introducing revised knowledge through the collection rather than relying entirely on model retraining. For a KMS, the intended benefit is an explanation grounded in applicable sources, while the original documents remain available for inspection.

The unit of retrieval affects what evidence reaches the generator. Splitting a document into passages, often called *chunks*, permits focused retrieval but can separate an instruction from its preconditions. Retrieving an entire document preserves more context at the cost of additional irrelevant material. RAGAs treats context relevance and answer faithfulness as separate concerns because finding useful information and using it correctly are different problems.[5] A citation consequently needs more than a matching document title: the cited passage must support the associated statement, including any qualifications that change its operational meaning.

2.4. Related cybersecurity systems

MoRSE addresses broad cybersecurity question answering through specialised retrievers over structured and unstructured knowledge. Its evaluation covers 600 questions concerning general cybersecurity, multi-document reasoning, and vulnerabilities, with expert-validated reference answers and model-assisted assessment. The study reports advantages over its selected language-model comparators.[18] Its contribution is evidence that organising heterogeneous cybersecurity sources around a query can improve answers in that benchmark. The evaluation unit remains the answer, however, rather than an analyst's completed incident-response task. The result therefore supports investigation of retrieval mechanisms without establishing how much time a responder would save.

GenDFIR concentrates on incident timeline analysis. It converts event data into a structured textual knowledge base, retrieves relevant events, and uses a language model without task-specific fine-tuning to produce an interpreted timeline. Its evaluation uses synthetic cyber incidents and task-specific assessments of evidence retrieval and resulting analysis.[11] Unlike a reference-document assistant, it retrieves observations belonging to the investigation itself. This makes event coverage and chronological relationships central to the task. Controlled scenarios allow the expected evidence to be specified, but performance on them leaves uncertainty about incomplete, inconsistent, or unfamiliar

evidence in operational investigations.

RAGIntel uses MITRE ATT&CK knowledge for attack-investigation questions and reports performance comparable to standalone language models in its evaluation.[2] This is a useful qualification to MoRSE's positive comparison: a retrieval stage can provide an explicit knowledge source without necessarily improving aggregate answer scores. Differences in source coverage, question type, and generator make the studies informative comparisons of approaches, not a ranking of systems under common conditions.

ForensicLLM investigates local model adaptation for digital-forensics questions. It fine-tunes a model with reduced-precision weights using question–answer material derived from research articles and curated digital artefacts. The study compares the resulting configuration with a base model and a retrieval-augmented baseline, reporting stronger correctness and relevance alongside professional assessments of the responses.[17] This comparison shifts attention from the retrieval mechanism alone to the complete model configuration and its domain preparation. Its local deployment also raises a distinct consideration: the feasibility of using specialist assistance where forensic information cannot be sent to an external generation service.

Table 2.1: Knowledge sources and evaluation focus in selected cybersecurity studies.

Study	Knowledge used	Main evaluation focus
MoRSE[18]	Heterogeneous cybersecurity references.	Answer quality across general, multi-document, and vulnerability questions.
GenDFIR[11]	Events from the incident under investigation.	Evidence retrieval and timeline analysis in synthetic incidents.
RAGIntel[2]	Structured MITRE ATT&CK intelligence.	Responses to attack-investigation questions compared with standalone models.
ForensicLLM[17]	Forensic research and curated artefacts used for domain adaptation.	Question answering, source attribution, and professional response assessments.

Evidence from malicious-package detection further limits general claims about RAG. Ibiyo et al. find that their tested knowledge bases do not improve classification performance, while their fine-tuned condition performs substantially better.[7] That task predicts whether code is malicious; it does not ask whether an explanation helps an analyst use response guidance. The different evaluation targets in Table 2.1 also explain why baseline selection must follow the decision being supported. Model choice, adaptation, retrieval, and access to additional knowledge are separate variables whose contributions cannot be inferred from the label "RAG" alone.

2.5. Reliability, confidentiality, and adversarial content

Retrieval introduces evidence into generation, but it also introduces an attack surface. Greshake et al. demonstrate indirect prompt injection, in which attacker-controlled content encountered by an LLM-integrated application contains instructions intended to redirect its behaviour. Their experiments include attempts to manipulate outputs and disclose information.[6] A cybersecurity report can therefore be useful evidence while also carrying hostile instructions. Source content needs to be treated as material to assess, not as authority to redefine the task or grant access. Separating instructions from evidence is a necessary defensive objective, but formatting or prompt wording alone does not establish resistance to these attacks.

Ordinary knowledge errors create a related reliability problem. An answer can reproduce an outdated source faithfully, omit a procedural exception, or express an uncertain incident hypothesis as fact. The Retrieval-Augmented Generation Benchmark (RGB) tests noise robustness, rejection when evidence is insufficient, integration of information, and robustness to false information; the evaluated models show particular difficulties with the latter three abilities.[3] For incident guidance, uncertainty and conflicting evidence therefore deserve explicit treatment rather than being hidden behind a single fluent recommendation.

Confidentiality concerns the complete information flow. Restricting document access is ineffective if restricted content can still reach an unauthorised answer, and a local generator does not by itself determine where embedding, retrieval, or evaluation services process data. Combining the trust concerns identified in CTI sharing with the disclosure risks demonstrated by prompt injection suggests treating authorisation as a constraint on eligible evidence, not a preference inferred by a language model.[1, 6] Source traceability then operates within that constraint: an explanation should expose the evidence a responder is permitted to inspect without disclosing protected material.

2.6. Evaluating knowledge access and response decisions

A KMS evaluation needs to distinguish finding knowledge, explaining it, and using it in a decision. At the retrieval level, precision measures the proportion of returned material judged relevant, while recall measures how much of the annotated relevant material was recovered. Rank matters because evidence placed far down a result list may be costly to locate. Passage-level assessment also differs from document-level relevance: a document can concern the right incident type while the selected passage omits the required procedure. For access-controlled collections, the relevant set must be defined for the requesting identity and the temporal scope of the question.

RAGAs separates context relevance, answer relevance, and faithfulness. Its faithfulness method decomposes a response into statements and uses a language model to assess whether the context supports them.[5] ARES trains lightweight judges for related dimen-

sions and combines their predictions with human annotations through prediction-powered inference to estimate system performance with confidence intervals.[16] These methods support scalable comparison, but their agreement with expert judgements needs checking in the target domain. Faithfulness to an incorrect source is not factual correctness, and factual correctness alone does not establish that a proposed action is suitable for a particular environment. Citation support and operational applicability consequently require separate examination.

Negative cases help reveal failures obscured by answerable questions. Following the distinctions examined in RGB, an assessment can include irrelevant passages, contradictory evidence, and requests for which the accessible collection contains no answer.[3] Appropriate abstention means recognising insufficient evidence; an unnecessary refusal when relevant evidence is available instead indicates a failure of retrieval or generation. Temporal cases add another distinction: a historical account may correctly cite superseded guidance, whereas a request for current response advice requires currently applicable knowledge. These cases test the knowledge lifecycle as well as the model's language behaviour.

For human outcomes, response quality and task efficiency need joint measurement. A search-only condition can establish how analysts perform when they inspect sources directly. Generation without retrieval tests the contribution of model knowledge, while generation with retrieval tests the additional value of contextual evidence. Comparisons are interpretable when the cases, available knowledge, and model conditions are controlled. Elapsed time should cover the response task rather than only model latency. Independently assessed decision correctness, severity of errors, and source consultation help distinguish useful acceleration from faster acceptance of unsupported advice. Usability and confidence capture the responder's experience but do not replace those outcome measures.

The organisational perspective extends evaluation beyond an individual answer. Yeboah's separation of knowledge-sharing processes and outcomes suggests examining whether new lessons are contributed and subsequently reused.[20] Relevant KMS measures include the time required to make approved knowledge available, whether corrections displace obsolete advice, and whether contributors can identify material needing review. A system that answers a fixed benchmark well may still lose value as its collection changes. Evaluation therefore benefits from both controlled retrieval tasks and observation of how knowledge remains usable through contribution, revision, and reuse.

2.7. Synthesis and research gap

The reviewed studies connect incident-response knowledge to several complementary research problems. Knowledge-sharing research explains why capture and maintenance depend on people and organisational purpose. Learned representations support access across differences in wording, while RAG offers a way to condition explanations on retrievable

sources. Cybersecurity evaluations establish task-specific benefits and limitations, with no uniform advantage for retrieval across the compared settings.[2, 7, 8, 18]

The remaining question is how these mechanisms support the complete path from maintained knowledge to a defensible incident-response decision. We therefore frame RQ1 around the knowledge and lifecycle needed for reuse, RQ2 around relevant retrieval and traceable guidance, and RQ3 around retrieval quality, response quality, task efficiency, and usability under confidentiality constraints. This framing treats optimisation as reduced effort accompanied by sounder decisions, not as output speed alone. Chapter 3 develops the design from these requirements.

CHAPTER 3

Sentinel KMS: Design and Development

3.1. Design purpose and use cases

We developed Sentinel as a knowledge-management system that uses machine learning to support incident analysis and response. It brings together a document library, source ingestion, and a conversational assistant. The purpose is to make applicable knowledge easier to locate, interpret, and reuse when an analyst must decide what to investigate or do next. Embedding models help retrieve relevant passages; a language model uses those passages to explain response options. Publication controls and inspectable evidence connect this assistance to maintained organisational knowledge.

Consider an analyst receiving the report, "I entered my password after following a suspicious message." The useful knowledge may be distributed across a phishing playbook, account-recovery guidance, and a local procedure assigning responsibility for identity administration. Sentinel provides two entry points: the analyst can browse the library directly or describe the situation to the assistant. The assistant receives the observations, retrieves accessible material, and is instructed to distinguish reported facts from possible explanations. The analyst can then open the supporting passages before deciding whether the advice fits the affected service and their authority to act.

This example defines the system's advisory scope. Sentinel supports the interpretation of incident information and the preparation of a response; the responder remains responsible for decisions and changes to operational systems. Knowledge custodians maintain the material on which that assistance depends. Editors propose revisions, administrators control publication, and operators monitor ingestion and service availability. Together, these roles connect knowledge capture and maintenance in RQ1 to contextual, traceable guidance in RQ2. RQ3 concerns how well that design supports retrieval, response quality, task efficiency, and usability.

3.2. Requirements and design priorities

The initial requirements work combined a literature review with semi-structured interviews with a research specialist to identify use cases and user roles. We develop that starting point into four functional requirements and three non-functional requirements in Table 3.1. The functional requirements follow the movement of knowledge from maintained content to incident guidance. The non-functional requirements apply through-

out that movement: access restrictions must survive processing, consequential operations must remain reviewable, and deployment choices must accommodate different hosting constraints.

Table 3.1: Requirements guiding the design of Sentinel.

Requirement	Intended capability	Design response
F1 Governed knowledge	Capture, organise, and maintain material suitable for reuse.	Classified articles, immutable revisions, review, publication, and expiry.
F2 Recoverable ingestion	Preserve submitted sources and resume interrupted processing.	Retained content, durable jobs, retry, and protected index replacement.
F3 Contextual retrieval	Locate accessible knowledge relevant to the incident question.	Embeddings, lexical search, indicator lookup, and rank fusion.
F4 Traceable incident guidance	Explain response options with inspectable supporting material.	Source-based prompts, playbook fallback, citations, and passage viewing.
N1 Access and disclosure safety	Respect knowledge restrictions throughout assistance.	Membership and lifecycle checks before retrieval, disclosure, and evidence viewing.
N2 Reviewable operations	Retain information needed to investigate system decisions and failures.	Revision history, job states, source references, and redacted audit events.
N3 Deployment flexibility	Separate infrastructure and model-hosting choices.	Configurable database, identity, chat, and embedding services.

Requirements specify the capabilities we seek; validation criteria specify how we judge them. For example, F2 calls for recoverable ingestion, while a corresponding criterion asks whether recovery uses the retained source and prevents an obsolete worker from replacing newer content. F3 requires contextual retrieval; its evaluation concerns the relevance of returned passages for defined questions. F4 requires traceability, with separate assessment of citation resolution and whether cited passages support the advice. Chapter 4 connects these criteria to technical checks and the evaluation of analyst-facing tasks. This distinction keeps design choices separate from observed effectiveness.

3.3. Architecture and allocation of responsibilities

Sentinel uses a React interface, a Spring application, and PostgreSQL with pgvector for vector storage. Figure 3.1 shows how these parts connect. The interface presents knowledge and captures requests. The application coordinates publication, ingestion, retrieval, and generation. PostgreSQL retains the knowledge, access rules, processing state, and evidence references needed by those activities. Identity and model services remain separately configurable dependencies.

We place publication rules and assistant orchestration in the same application so that a retrieved passage follows the same access policy as an article opened in the library. Keeping relational and vector data together also allows retrieval to check publication and membership alongside candidate selection. This choice reduces the number of services needed to coordinate evidence eligibility. Its cost is tighter coupling between search and the knowledge schema: changes to classification or lifecycle rules must be reflected consistently in direct reading and retrieval.

The data model separates representations with different purposes. An article provides the current reading view. A revision preserves a particular content state. An evidence snapshot retains the text against which a citation is resolved. Embeddings are derived search representations and can be regenerated without redefining the underlying knowledge. This separation allows an editor to revise guidance while preserving the identity of material previously used in a conversation. A separate assertion store records evidence-linked entities and relationships; the assistant retrieves passages rather than traversing that store.

Long-running work is handled asynchronously. Admission records a source before an ingestion worker extracts and indexes it. Publication records a reindex request in the same transaction as the knowledge change. The browser can therefore distinguish acceptance from subsequent processing, while a restart can recover outstanding work. We accept a delay between publication and search availability in exchange for keeping model calls outside the publication transaction and protecting the identity of the revision being indexed.

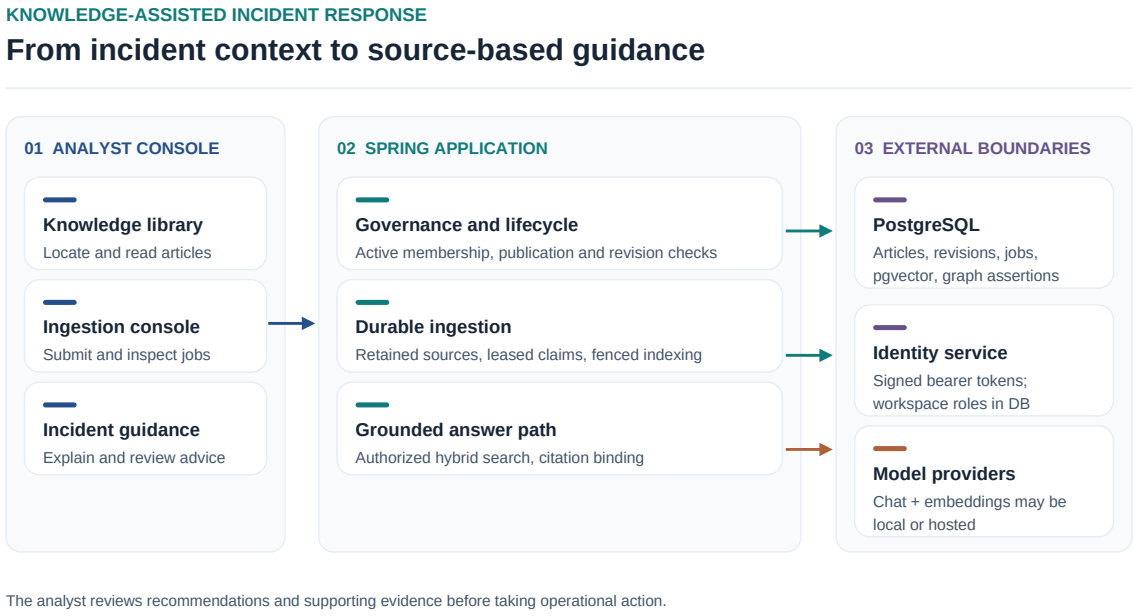


Figure 3.1: Sentinel's knowledge, application, and external-service boundaries.

3.4. Knowledge organisation and maintenance

3.4.1. Articles and curated incident playbooks

The library organises articles by category, severity, tags, source, and update date. Article bodies contain titled sections, with related documents and indicators of compromise where applicable. These fields serve different reading tasks: a category narrows the type of knowledge, a source identifies its origin, and related documents connect an initial observation to further investigation. Severity describes the material's assigned importance; it does not diagnose the incident described by an analyst.

We include a curated collection of response playbooks to give the assistant procedural material alongside advisories and technical descriptions. Its scenarios include phishing, ransomware, account takeover, exposed credentials, application compromise, data exposure, and recovery. Each playbook follows a recognisable structure: applicability, questions to answer, first actions, evidence to preserve, containment and recovery, verification, and escalation. Decision checkpoints explain where context changes the response. For example, the phishing guidance distinguishes receiving a message from entering credentials or executing a downloaded file. We include these distinctions so that retrieved material can address the reported interaction rather than describe the threat only.

The playbooks are source-attributed editorial syntheses, identified as AI-assisted and requiring human review. Their admission checks require attribution, declared categories, consistent identifiers, and the review notice; they also exclude unverified indicators from this general guidance. Installation adds missing articles through the governed publication process and preserves existing content when it differs. A custodian must review such differences rather than lose local adaptations during a refresh. Here, approval permits reference use; operational applicability still depends on review in the receiving organisation.

3.4.2. Review, publication, and reuse

An editor can create a draft revision and submit it for review while the current approved article remains available. Administrative approval rejects self-approval and publishes the accepted revision. Rejection leaves the existing publication intact. State-version checks detect a decision based on an outdated workflow state, which matters when several custodians are maintaining the same procedure. This workflow supports knowledge improvement without making unfinished advice available to the assistant.

Supersession publishes a reviewed successor, whereas administrative rollback creates a new revision from previously published content. Rollback preserves a forward-moving revision history and uses a separate administrative publication route. Expiry removes material from present read eligibility while retaining its history. These operations make maintenance part of knowledge reuse: a response procedure can remain attributable without remaining suitable for current advice.

The publication transaction also records durable reindex work. Replacement is tied

to the intended revision and governance state, so a delayed worker cannot publish search material for an obsolete state. During reindexing, an approved revision may temporarily lack searchable embeddings. We favour this gap over allowing superseded guidance to enter a new answer simply because its vectors are still stored.

We keep saved answers separate from published guidance. A conversation can support follow-up work, while knowledge custodians decide which lessons belong in a revised article. This separation prevents a generated response from acquiring publication authority merely because it has been retained. Reuse therefore depends on editorial judgement as well as the ability to search prior knowledge.

3.4.3. Maintaining externally supplied knowledge

Source connectors complement local material with vulnerability and incident information. Refreshes for NVD, CISA KEV, and SANS ISC use persisted work queues, progress records, and source checkpoints. Unchanged records retain their revision; changed source-owned records receive new revisions and evidence. Manual policy or content changes require review instead of being silently overwritten by a feed. Fetching and reindexing remain separate stages, so connector progress and search readiness answer different operational questions.

Trusted producers can publish reference material through an explicit admission route without manual review of each item. This is a decision about which producers the KMS accepts, with the corresponding maintenance responsibility assigned to its operators. Together with the curated collection and editor-managed articles, it gives the library several acquisition routes while retaining a common publication and evidence model.

3.5. Recoverable ingestion of local and remote sources

Incident knowledge also arrives as uploaded documents or material imported from a URL. Sentinel checks uploads for accepted formats and size, assigns source identities, and retains their content before exposing a queued job. URL admission checks the destination; the worker retains the first successful response before extraction. A retry consequently processes the same retained material rather than relying on an open browser session or downloading an unnoticed replacement. Optional owner-scoped request identifiers allow repeated submissions of the same request to recover its existing job.

Processing separates extraction, sanitisation, chunking, embedding, and indexing. For uploaded text, the configured chunker uses character windows of 1200 characters with an overlap of 150. Overlap retains some context across boundaries, while fixed windows keep processing predictable across document formats. The tradeoff is that a window may divide a sentence or procedure. Governed articles instead provide retained evidence passages with explicit identities. Missing source content or empty extraction produces a failed job that can be inspected and, when appropriate, retried.

A worker holds a time-limited claim with a token and processing generation. Progress updates and index replacement must match that live claim. Recovery advances the generation, preventing a delayed worker from overwriting a later retry or restoring deleted material. Repeated embedding calls remain possible when a worker loses its claim after contacting a provider; the protected operation is the replacement of searchable content. This design addresses F2 by preserving continuity of knowledge acquisition through interruptions.

An indexed upload retains its own access scope and processing history. Indexing alone does not turn it into an approved library article. The assistant can use an authorised upload as grounding, but article publication and article-level citations require their corresponding evidence identities. Keeping these routes distinct allows an analyst to consult submitted material without making it organisational guidance by accident.

3.6. Machine-learning retrieval and incident guidance

The assistant follows retrieval-augmented generation (RAG): it retrieves knowledge for the reported situation and supplies that material to a language model. We use machine learning for representation and explanation, while explicit application rules determine which knowledge is eligible and how its use can be inspected.

3.6.1. Combining semantic and lexical retrieval

Machine learning enters retrieval through embeddings: numerical representations of passages and questions produced by an embedding model. Sentinel compares the question representation with stored passage vectors using cosine distance. We use this route to address differences between the language of a report and the terminology of the library. A person may describe encrypted files without naming the relevant response category. Lexical search supplies a complementary route for terms and identifiers that appear explicitly in the question and stored text.

Both searches apply article eligibility and access conditions before limiting their candidate lists. Eligible article passages belong to the current approved, active, unexpired revision and match a retained evidence span. Upload candidates follow their durable-source access rules. Filtering at this point allows accessible material to compete for the available positions instead of first filling them with inaccessible matches. If vector retrieval fails, lexical retrieval still applies the same access conditions.

We combine the two ranked lists using reciprocal-rank fusion. For passage d, the ordering score is

$$F(d) = \sum_{\ell \in \{\text{vector,lexical}\}: d \in \ell} \frac{1}{60 + \text{rank}_\ell(d)}. \tag{3.1}$$

Ranks begin at one. This choice combines positions rather than treating cosine similarity and lexical relevance as directly comparable quantities. A passage found by both

methods accumulates a contribution from each list. The score determines ordering, not the probability that advice is correct. The configured minimum similarity applies to vector candidates, while category and minimum-severity filters use article metadata in both searches.

3.6.2. Establishing the context for an answer

The assistant supplements hybrid retrieval with access-aware indicator lookup. This gives literal technical observations a route to matching knowledge alongside the broader semantic search. Indicator lookup uses the original question and is separate from the hybrid search's category and severity filters. Its evidence span identifies the indicator occurrence; a surrounding summary still needs to be checked against the full article. After combining candidates, the application resolves readable articles and exact evidence references before constructing model context. An optional reranker receives only authorised candidates.

Conversational rewriting can turn a follow-up question into a standalone search question using recent turns. We place this step after literal-question retrieval has established usable evidence. If that initial retrieval is empty, the assistant returns a local message directing the analyst to refine the question or browse playbooks. This conservative ordering avoids a generated answer detached from accessible knowledge, although it can also leave a context-dependent follow-up unanswered. The question may already have reached the embedding provider during retrieval; the gate controls the subsequent rewriting, reranking, and answer-generation stages.

The prompt combines the question, bounded recent conversation, and selected sources. Each article passage carries source information and a server-issued evidence token. The source-context budget limits prompt growth, at the cost of potentially truncating the final passage. We therefore treat retained grounding references as the material selected for context, rather than a record of the model's reasoning. Knowledge remains external to the generation model: revisions can change what is retrieved without retraining it.

3.6.3. Turning retrieved knowledge into response guidance

We shape the generation instructions around the decisions an incident responder faces. For suspected incidents, the model is asked to separate observations from explanations and organise supported advice around immediate actions, investigation and evidence preservation, recovery, and verification. The instructions call for an explanation of each action's purpose, expected result, and responsible actor. They limit clarification to three focused questions and prohibit assumptions about the operating system, affected platform, or responder's administrative access.

The instructions also require the model to explain operational impact before disruptive actions and to preserve evidence before cleanup. Requests for redacted observations replace requests for passwords, secret keys, verification codes, or complete sensitive logs. A conceptual question can receive a short explanation instead of a forced incident checklist.

These instructions define the intended assistance; the reviewable sources and subsequent answer assessment remain necessary because a language model can misapply them.

The application supports structured answer blocks and a natural-language conversation route. Both use the same practical guidance and authorised context. The model is instructed to copy evidence tokens supplied by the server. Response processing resolves accepted tokens to retained references and removes unsupported citation tokens. Article citations and exact-passage references are then associated with the assistant message, making the advice available for later inspection.

3.6.4. Useful behaviour when generation is unavailable

A model failure need not remove access to existing guidance. When generation fails or is temporarily blocked, Sentinel first looks for an eligible playbook among the retrieved articles. It independently resolves up to three bounded sections titled *First actions*, *Questions to answer*, and *Evidence to preserve* from that playbook's current approved revision. The section text comes from retained evidence, with a separate access check for each passage, rather than from trusting the retrieval metadata alone.

The fallback quotes these sections and labels them as existing playbook guidance, not a personalised answer. If section expansion cannot be authorised, it provides available retrieved article references for direct reading. It does not combine fragments into a locally invented response plan. This behaviour preserves a useful part of the KMS during a generation outage while making the loss of contextual adaptation visible. The same final authorisation and citation-persistence checks apply to fallback and generated responses.

3.6.5. Keeping guidance attached to evidence

An exact article citation identifies the article, revision identity and number, evidence snapshot, and passage boundaries. Retrieval checks the passage against the retained snapshot. Figure 3.2 follows this relationship through to inspection. An article title alone would be insufficient because the displayed article could change after the answer was produced. Retaining a passage identity makes that change distinguishable from the evidence originally selected.

Before releasing an answer, the application checks that its grounding remains authorised and persists its references. The conversation route buffers generated output before sending its server-sent events, allowing this final disclosure check to occur before text reaches the browser. The tradeoff is a longer wait for the first visible answer rather than immediate provider-token display.

Evidence inspection performs another current-access check. Superseded, expired, revoked, or otherwise unavailable references return an unavailable state, while their identities can remain in the conversation record. We prioritise present disclosure rules over unrestricted replay of previously visible passages. A resolved citation establishes which

material can be inspected; deciding whether that material supports the recommendation remains an analyst and evaluation task.

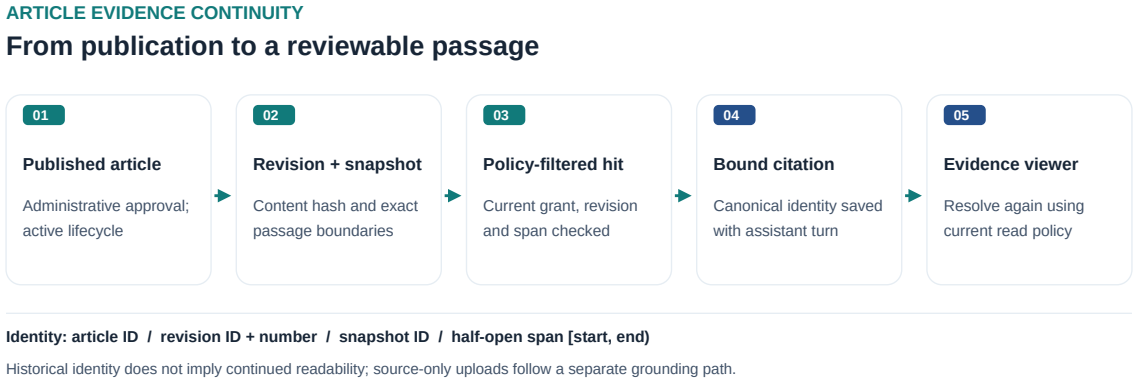


Figure 3.2: A governed article's path from publication to access-checked evidence inspection.

3.7. Analyst interaction and knowledge reuse

The interface connects direct reading with conversational assistance. In the knowledge library, topic shortcuts lead to playbooks for recognisable problems such as suspicious messages, encrypted files, compromised accounts, and exposed secrets. Search, category and severity filters, and sorting support more deliberate exploration. An article opens in a reading pane with its sections, source information, and related documents. For playbooks, an action transfers a request to the assistant to help apply the selected guidance, asking about the situation and actions already taken.

The assistant accepts free-text questions and provides an optional response-plan intake. The intake asks what happened, which devices, accounts, or services are affected, and what the reporter has already done. It turns those observations into a structured question requesting priorities, evidence preservation, recovery, verification, and escalation. Unknown fields remain explicitly unknown. This design gives the retrieval and generation stages more context without forcing the reporter to classify the incident before asking for help.

Saved conversations support follow-up and handover; answers can be copied, exported, and rated for usefulness. The response separates article navigation from exact-evidence inspection. Opening a source provides broader reading, whereas the passage action opens the cited revision and span. The dialog checks that the returned identity matches the request and manages keyboard focus while open. This distinction supports both understanding the procedure and checking the specific material associated with an answer.

The ingestion view presents source submission, processing stages, job status, retry controls, and connector refresh. These controls expose the difference between adding

material and having it ready for retrieval. Figures 3.3–3.6 illustrate selected interface states using the actual frontend with deterministic mocked authentication and application responses. Their article text, answers, passages, and counters are demonstration data, not corpus measurements, model results, or observations from an analyst study.

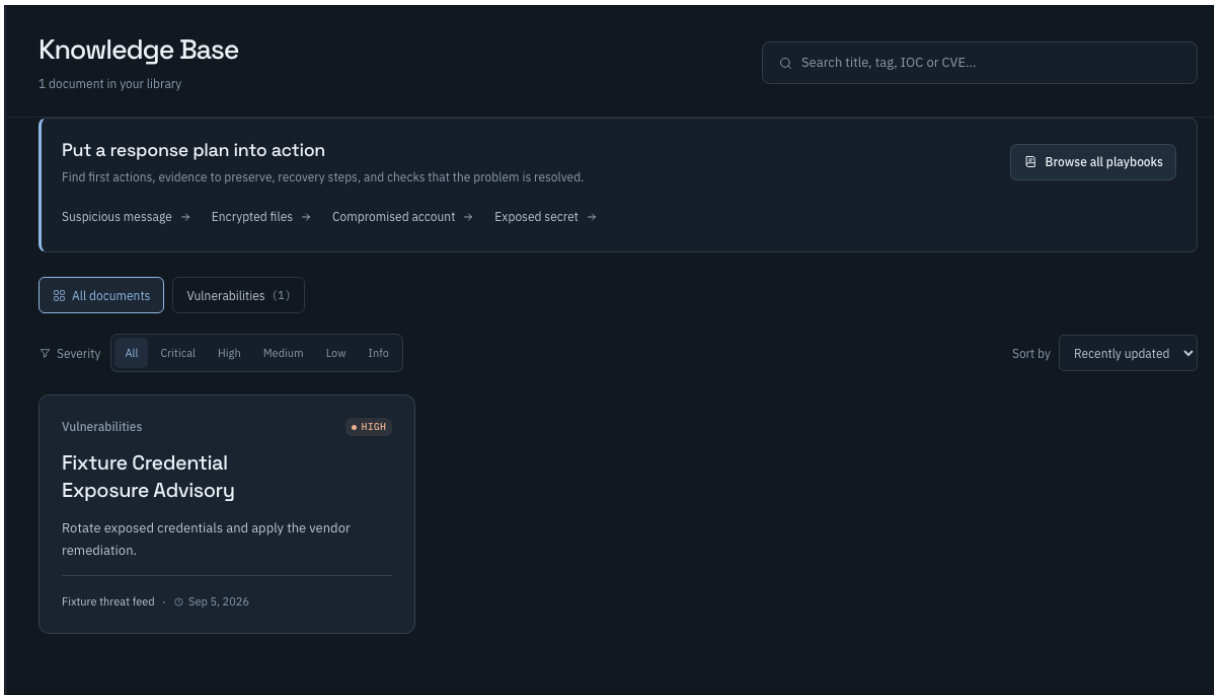


Figure 3.3: Knowledge-library filters and problem-oriented playbook browsing (mocked data).

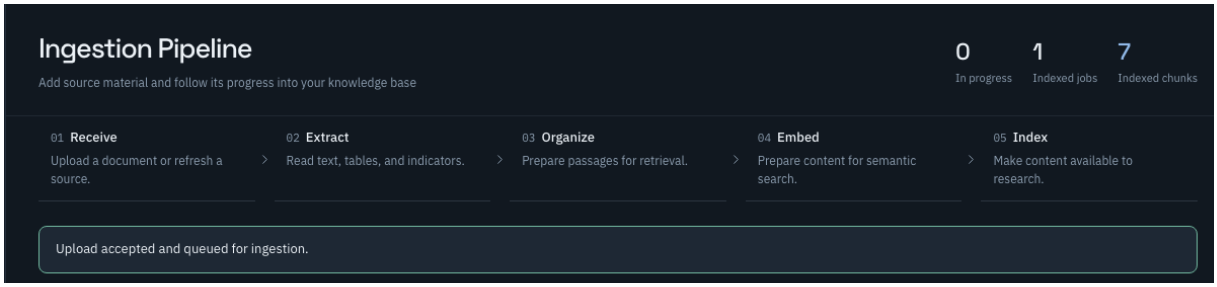


Figure 3.4: Ingestion stages and source-submission acknowledgement (mocked data).

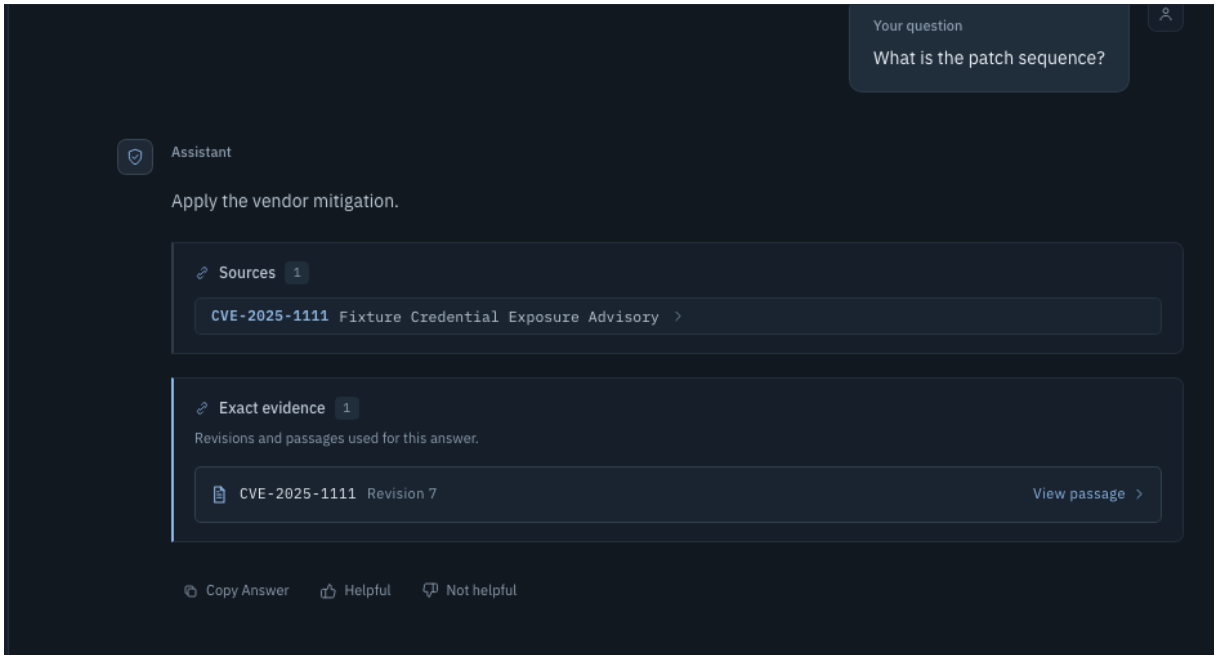


Figure 3.5: Assistant answer with article and exact-passage actions (mocked data).

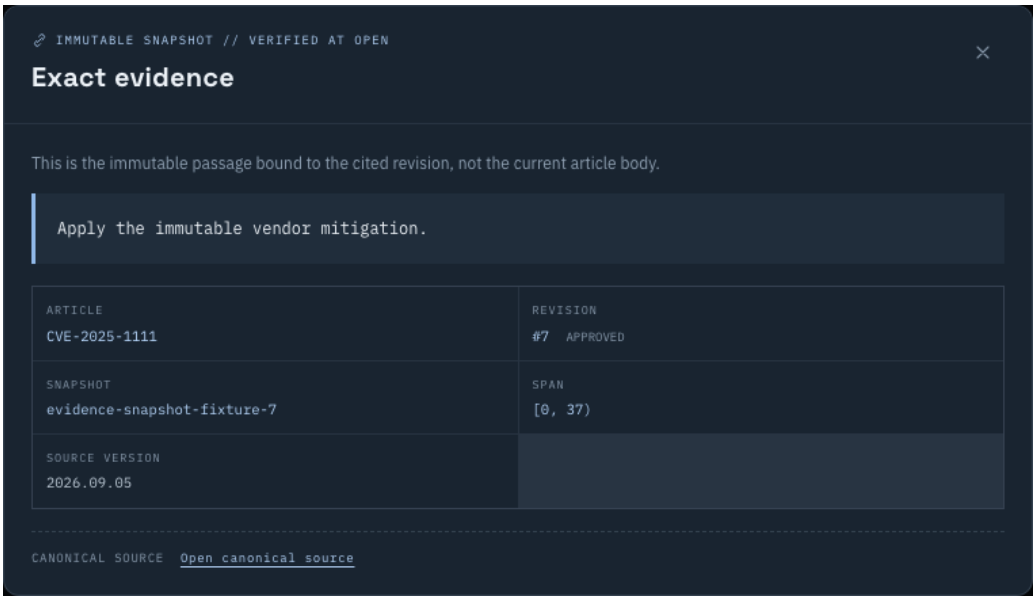


Figure 3.6: Evidence dialog identifying the cited revision and passage (mocked data).

3.8. Confidentiality and operational choices

Access policy supports knowledge sharing at different levels. Public articles require authentication; internal material requires workspace membership; confidential and restricted material require editor and administrator authority respectively. Active database memberships determine those roles. Missing or inconsistent governance results in denial. These controls apply to direct reading, retrieval, and evidence disclosure, so a conversational request does not provide an alternative route around the library's restrictions.

Retrieved content is treated as untrusted data. Sanitisation targets recognisable injection patterns before uploaded text is indexed and before source text enters a prompt. The prompt distinguishes source material from instructions. These defences reduce exposure to known patterns but require adversarial evaluation, particularly when unfamiliar documents are admitted. The separation of source permissions from text and metadata remains necessary even when sanitisation leaves a passage apparently harmless.

For N2, revision and workflow histories describe knowledge changes, while job records explain processing progress. Redacted audit events record consequential operations and outcomes. Successful transactional changes produce their success events after the transaction completes. Audit persistence is best-effort: an audit failure is logged without replacing the original operation's result. This favours service continuity but leaves a completeness limitation that operators must consider when investigating an incident involving the KMS itself.

For N3, chat and embedding services can use hosted providers or local models through Ollama. Database and identity hosting are configured separately. Choosing local inference therefore addresses model processing, while the complete disclosure boundary also depends on where knowledge and identity services run. Hosted embedding requests transmit passage or question text; hosted answer requests transmit the question, selected context, and recent conversation. Access eligibility within the KMS and permission to send material to a provider are separate deployment considerations.

The vector store expects 1024-dimensional embeddings. Changing the embedding model requires compatible representations and reindexing, with a storage change if dimensionality differs. We keep these choices explicit because confidentiality, provider availability, and reproducibility depend on the whole deployment rather than the chat model alone.

3.9. Chapter synthesis

Sentinel connects maintained knowledge to incident-response work through recoverable acquisition, contextual retrieval, and source-based guidance. The curated playbooks supply procedural content; embeddings and lexical search locate candidate passages; generation adapts the explanation to the reported situation. Direct reading, fallback guidance, and exact-evidence inspection keep the knowledge usable beyond a single model response. The

resulting design addresses F1–F4 while treating access safety, reviewability, and deployment flexibility as shared constraints. The following chapter examines the system against these requirements and the research questions, separating technical behaviour from the quality and usefulness of the assistance.

CHAPTER 4

Evaluation Design and Current Evidence

4.1. Purpose and evidential boundary

The evaluation must distinguish a functioning implementation from a useful and safe decision-support system. Software tests can establish that specified inputs produce specified outputs; they cannot establish that generated security advice is correct, that a citation entails a claim, or that analysts make better decisions. This chapter therefore separates executable regression evidence, inspection of evaluation mechanisms, and a prospective empirical protocol. The project's evaluation protocol informs the design, but its existence is not evidence that a study occurred.

The present contribution is an evaluation design backed by engineering verification of the inspected, modified working tree on 29 September 2026. The current checks exercised a synthetic-report quality gate, Java tests including isolated container-backed cases, frontend unit tests, scripted Chromium journeys, and a production frontend build. An earlier audit also exercised the same report validator, but its result is not added to the current total. Neither execution is a live retrieval benchmark, a provider comparison, an expert annotation exercise, or an analyst study. Accordingly, no measured model ranking, improvement in task completion, participant count, or significance claim is reported. Missing empirical results remain missing; engineering thresholds and synthetic inputs are not substitutes. Appendix A explains the evidence classes and the external verification records retain execution details.

The implementation-led questions in Chapter 1 determine the evidence boundary. RQ1 is addressed by tracing revision, passage, policy, and lifecycle identities across the source definitions described in Chapter 3. RQ2 is addressed by distinguishing the assistant's runtime grounding and citation path from the narrower hybrid-search evaluator and from capabilities not connected to generation. RQ3 requires examining stale-evidence, authorization, and hostile-content controls without promoting their presence into an operational-security guarantee. This chapter directly addresses RQ4 by separating executed validator checks from unexecuted mechanisms and the empirical work still required.

That subsequent empirical work has three dimensions: whether retrieval delivers the authorized evidence needed for a security question; whether generation preserves its meaning, scope, and temporal qualification; and whether assistant-supported work improves analyst decisions without increasing unsafe reliance. These are proposed effectiveness

questions, not replacements for the implementation-led research questions or findings of this thesis. They require different units of observation: ranked evidence, generated claims, and analyst–case decisions. A successful result at one level must not be transferred automatically to another. The component distinction is informed by RAGAs and ARES; the authorization and proposed analyst-study requirements are specific to this project.[5, 16]

4.2. Available artifacts and executed checks

4.2.1. Datasets and evaluation mechanisms

The retrieval evaluation dataset contains ten cases, including severity- and category-scoped questions. Expected relevance is expressed as article identifiers. The separate answer evaluation dataset contains nine synthetic cases: two answerable cases and one each for unanswerable, ambiguous, conflicting, stale, multi-document, adversarial, and authorization scenarios. Temporal disclosure is represented within an answerable case. Both the answer dataset's overall review record and all nine case records remain pending, with no reviewer identity or review timestamp supplied.

These are regression fixtures, not a representative incident sample. Their expected phrases and evidence identifiers encode developer assertions. In particular, the stale fixture requires abstention even though its question asks about a superseded snapshot; whether that is the appropriate response to a historical question requires rubric review. The fixture must not silently become a normative rule that historical evidence can never be discussed.

The evaluation mechanisms separate retrieval scoring, deterministic answer checks, saved-output comparison, optional provider execution, and external annotation import. The answer checker examines phrases, citation provenance, temporal markers, and forbidden identifiers. Its explicit heuristic labels are essential: matching a required phrase is not semantic correctness, and omission of a forbidden citation identifier does not rule out uncited disclosure. Immutable revision and span fields support traceability but do not by themselves prove entailment.

4.2.2. What the offline run establishes

An earlier, isolated Python report-validator audit completed twelve tests successfully. On 29 September 2026, the same twelve-test report-gate suite also passed on the current modified working tree. The tests use temporary synthetic reports in subprocesses; neither application startup nor a live retrieval request is involved. The earlier pass is not an additional twelve current cases, and the current pass is not a component of the Java, frontend, or browser totals. Test elapsed time is suite duration, not retrieval or model latency.

The fixtures exercise acceptance of structurally healthy retrieval and JUnit reports, and rejection of low recall, empty reports, non-finite metrics, inconsistent totals, duplicate

identifiers, false pass declarations, skipped tests, missing required suites, and concealed testcase failures. Even the test named for an executed integration report uses a synthetic XML document. Its success establishes report-validator behavior, not that a database integration test ran. The numeric retrieval scores inside the test helper are likewise fabricated test inputs, never observations of the assistant.

The gate requires every declared retrieval case to pass and checks aggregate recall, MRR, and nDCG against defaults of 0.8, 0.6, and 0.6. It does not independently reconstruct those aggregate values from ranked evidence or bind a report to a frozen corpus. Its role is bounded validation of declared results. The report gate, including its current rerun, must not be conflated with the separate application suites below or with provider-backed retrieval assessment.

4.2.3. Executed software verification and scripted walkthrough

Table 4.1 reports the 29 September 2026 engineering checks against the modified working tree, not the committed baseline. A green test outcome means the asserted software behaviour held for the tested inputs and environment; it is not a sampled human outcome or a measure of generated-answer accuracy. The Java suite includes isolated container-backed PostgreSQL checks alongside mocks, but no production database or hosted model was involved. At least the illustrative screenshot journey in the Chromium suite intercepts authentication and application responses; it does not run the complete ingestion-to-generation path.

Table 4.1: Observed engineering verification on the modified working tree, 29 September 2026; test counts are not participants or evaluated security questions. The report gate is a distinct synthetic-check layer, not part of the application-suite totals.

Layer	Observed result	What this result cannot establish
Report quality gate	12 passed on current rerun	Fabricated retrieval and test reports exercise validator logic, not live retrieval quality.
Java tests	408 in 88 reports; 0 failures, errors or skips	Isolated containers and mocks do not certify a hosted deployment or expert answer quality.
Frontend unit tests	91 passed in 13 files	Component behaviour in a test DOM does not prove backend authorization.
Chromium browser tests	17 passed	The mocked capture demonstrates an interface contract, not real provider, authentication or database behaviour.
Frontend production build	Succeeded; 133 modules transformed	Compilation alone does not verify live integration or answer quality.

For the illustrative figures, a scripted analyst persona follows a fixed sequence: sign in through intercepted test authentication, upload a fixture document, ask a prepared mitigation question, and open the article-revision passage. The browser asserts the upload

status, cited answer, exact evidence text, canonical-source link, and request counts before recording the figures. The uploaded file does not cause the cited answer: intercepted application responses independently supply a fixed article and answer. Other contract cases exercise unavailable evidence, job retry, watchlist creation, expired sessions, and permission denial. These are simulated interactions, neither human-subject interviews nor evidence that an analyst understood or trusted the interface appropriately. The displayed indexed state in the capture is returned by a mock response, not by an ingestion worker.

The current Java test reports were counted by parsing 88 test-result documents: 408 tests, with no failures, errors or skips. Earlier test counts and a previous unsupported-runtime frontend failure describe a different verification snapshot and are not substituted for these current results. The report-gate rerun passed twelve of twelve fixture tests; the frontend test runner reported 91 passed tests across 13 files; the Chromium runner reported 17 passed tests; and the production build transformed 133 modules without error. The exact commands, environments, timestamps, and report artifacts belong in the external verification record. None of these counts is a live RAG benchmark.

4.3. Prospective corpus and comparison design

Before empirical execution, freeze the source documents, revision statuses, extraction and chunking configuration, embedding identifier, index snapshot, retrieval configuration, prompts, generation parameters, and access-policy snapshot. Preserve hashes and a manifest connecting case identifiers to authorized passages. A router alias alone is insufficient model identification when the selected underlying model can change; retain the actual identifier when available and disclose unresolved routing variability.

Construct development and held-out sets before tuning. Split related incidents and document families together so that paraphrases or adjacent revisions do not leak between sets. Include routine questions as well as difficult and negative cases, stratified by answerability, evidence multiplicity, recency, and authorization boundary. Domain reviewers must judge relevance against the evaluator's permitted corpus, not a global corpus containing inaccessible material. Otherwise an access-control success could be misclassified as retrieval failure.

Compare three conditions on identical case identifiers. Search-only exposes ranked authorized passages without generated synthesis. No-RAG uses the same generation model and response budget without retrieved organizational context. RAG adds the frozen retrieval context to that model. Search-only should be assessed through evidence finding and downstream task decisions; lack of generated prose is not an answer-quality failure. Local and hosted models form an additional comparison only for cases whose data policy permits both. Analyze the common eligible subset separately rather than presenting policy-driven exclusions as model superiority.

Record context and output budgets, temperature, stopping rules, prompt versions, and query-rewrite behavior. Evaluate optional reranking through a paired on/off abla-

tion that holds the candidate pool constant. A separate end-to-end comparison may include its additional latency and calls. The existing graph represents assertions, not an active graph-retrieval condition; neither graph-enhanced recall nor a graph ablation is currently available. Any future graph condition must first demonstrate which graph-derived evidence enters the response path.

Store raw outputs before scoring in restricted storage, with public reports carrying artifact bindings rather than sensitive answer text. The saved-output runner requires complete baseline case coverage, while provider execution requires explicit opt-in. These mechanisms facilitate comparisons but do not certify that supplied baseline outputs were generated under controlled conditions. In particular, the HTTP comparison adapter sends the question, baseline name, and fixture evidence to a caller-supplied endpoint for every baseline, including no-RAG. It does not itself invoke the application's assistant pipeline or enforce context removal for no-RAG. A controlled comparison therefore requires an independently verified endpoint implementation and request traces; a baseline label alone is insufficient. Retain execution manifests, failures, and configuration evidence outside the scoring report as necessary.

4.4. Metrics and operational definitions

4.4.1. Retrieval quality

Let R_q be the independently judged relevant article set for question q, and let $L_q = (d_1, \ldots, d_m)$ be the ordered, deduplicated retrieved articles. For cutoff $k > 0$, define $h_i = \mathbf{1}[d_i \in R_q]$ and $H_q(k) = \sum_{i=1}^{\min(k,m)} h_i$. The proposed precision and implemented recall definitions are

$$P_q@k = \frac{H_q(k)}{k}, \qquad R_q@k = \frac{H_q(k)}{|R_q|}. \tag{4.1}$$

Precision here treats unfilled slots as non-relevant; report the number returned alongside it. Recall is undefined for an empty relevant set in the scientific analysis, so those cases receive a separate negative-retrieval analysis. The implementation returns zero for empty relevance sets; that convention must not be mistaken for an expert judgment of unanswerability.

Define reciprocal rank and its case-wise mean as

$$\mathrm{RR}_q = \begin{cases} 1/r_q, & r_q = \min\{i : h_i = 1\} \text{ exists}, \\ 0, & \text{otherwise}, \end{cases} \qquad \mathrm{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{RR}_q. \tag{4.2}$$

The implementation scans the returned ranked list for reciprocal rank. With binary

relevance, normalized discounted cumulative gain is

$$\mathrm{nDCG}_q@k = \frac{\sum_{i=1}^{\min(k,m)} \frac{h_i}{\log_2(i+1)}}{\sum_{i=1}^{\min(k,|R_q|)} \frac{1}{\log_2(i+1)}}. \tag{4.3}$$

Again, the scientific analysis separates cases with a zero denominator. Macro-averages give each eligible question equal weight; stratified results expose concentrated failures that an overall mean could hide.

The retrieval evaluator invokes authenticated hybrid search with scope filters, deduplicates article identifiers, and marks a case as passing whenever recall exceeds zero. Thus its pass flag means at least one expected article appeared, not that every required source was retrieved. Chunk-level top-k selection precedes article deduplication, so the final list may contain fewer than k articles. Source-only upload hits are omitted from article-level scoring. The endpoint shares the hybrid-search component with the assistant, not its complete grounding pipeline: it does not add indicator-index matches, run conversational rewriting or reranking, perform the assistant's subsequent evidence-resolution steps, or assess generated answers. Precision, passage-level relevance, and independent-source diversity are proposed extensions, not existing reported endpoint metrics.

Execution permission is separate from document readability. The evaluation controller checks dataset viewing and retrieval execution against access-service grants and active workspace membership. Execution needs both dataset-view and execution grants in the same workspace, rather than merely an authenticated bearer token or an editor role. The evaluator then passes the requester's identity to hybrid search and enforces a configured maximum case count. These checks do not adjust the fixture's expected article identifiers to the evaluator's permissions. Before measuring retrieval, reviewers must therefore confirm that each expected set is appropriate for that identity; an operational grant does not make otherwise protected documents readable.

4.4.2. Answer quality and safety

Reviewers should segment an answer into material claims C_q and identify claim–citation links E_q. Let $s(c)$ indicate support by authorized, temporally appropriate evidence, and $e(c,p)$ indicate that cited passage p entails claim c. Define

$$\mathrm{Support}_q = \frac{\sum_{c\in C_q} s(c)}{|C_q|}, \qquad \mathrm{CitationAccuracy}_q = \frac{\sum_{(c,p)\in E_q} e(c,p)}{|E_q|}. \tag{4.4}$$

Also report citation coverage: the proportion of evidence-requiring claims linked to adequate supporting passages. Empty claim or citation sets are not automatically perfect; classify abstentions separately and mark inapplicable ratios explicitly. Correctness addi-

tionally requires completeness, appropriate scope, and absence of materially misleading omissions. A supported but incomplete answer can still fail the task.

Let a_q indicate required abstention and b_q observed abstention. Report both

$$\text{AppropriateAbstention} = \frac{\sum_q a_q b_q}{\sum_q a_q}, \qquad \text{OverAbstention} = \frac{\sum_q (1 - a_q) b_q}{\sum_q (1 - a_q)}. \tag{4.5}$$

Undefined denominators remain unreported with counts explained. Temporal correctness requires the response to distinguish current approval, historical state, uncertainty, and supersession, not merely repeat a date. Unsafe-action rate is the fraction of evaluable responses containing reviewer-confirmed harmful recommendations; report severe failures separately from minor errors. None of these expert-scored quantities is measured by the Python run.

For efficiency, collect elapsed wall-clock time, timeouts, token usage, and provider-reported cost for each attempted case. Distinguish time to first output from completed response latency. Report missing cost as unknown, not zero, and separate local inference resource consumption from provider billing. Availability and failure counts accompany quality summaries so that selective successful responses cannot inflate apparent performance.

4.5. Expert review and adversarial validation

Two qualified reviewers should independently validate the dataset and rubric, then review each baseline–case candidate without seeing the competing scores. Record reviewer identifiers, qualifications, dates, evidence inspected, rationales, and seven outcome labels corresponding to the annotation schema. Resolve disagreement or uncertainty through a separate adjudicator. Report pre-adjudication agreement and its denominator; adjudicated labels do not demonstrate that independent agreement was originally high.

The external annotation importer and scientific-completeness gate provide a procedural boundary. Reviews bind to dataset and output hashes, candidate digests, baseline identifiers, case identifiers, and immutable evidence tuples. The gate requires independent accepted reviews and appropriate adjudication; missing external review records cannot be replaced by fixture review metadata. However, structural checks cannot authenticate reviewer qualifications or prove actual independence. Those responsibilities remain organizational. A complete review containing failing quality labels is still a complete review, not evidence of satisfactory performance. All external reviews remain pending for the supplied dataset.

Adversarial evaluation should combine held-out malicious-source variants with benign controls. Test indirect instructions inside documents, forged authority statements, misleading citations, contradictory revisions, irrelevant retrieved passages, ambiguous scope, and attempts to obtain inaccessible evidence across users or workspaces. Use synthetic canaries rather than real secrets. Record whether protected content reached retrieval,

prompts, outputs, logs, or citation views; a clean final answer alone does not establish that access boundaries held.

The existing denied-reference fixture checks a forbidden identifier, while its adversarial fixture checks specific required and forbidden phrases. Both can miss paraphrased violations and can penalize harmless quotation. Their deterministic results therefore remain diagnostic. Real authorization validation requires an isolated deployment with distinct identities, database-authoritative permissions, request traces, and controlled provider-call observation. Safety testing must not execute operational recommendations against production assets. Preserve failed cases and stop a study on predefined severe disclosure or harmful-action criteria.

4.6. Statistical and ethical protocol

Pre-register a primary outcome and a practically meaningful effect before estimating sample size. A defensible pilot or external estimate should inform prospective power analysis; the repository supplies no justified participant count. For case-level model comparisons, retain pairing across identical cases and repetitions. Bootstrap uncertainty at the independent case or incident-family level, not at the level of correlated claims. Report effect estimates and confidence intervals alongside raw counts, with multiplicity control for confirmatory secondary comparisons and exploratory analyses clearly marked.

The analyst study requires institutional or organizational approval, informed consent, and a documented recruitment procedure with experience strata. Use a counterbalanced within-participant comparison of ordinary search and assistant-supported work, assigning different matched case sets to reduce recall effects. Record allocation seeds, task order, elapsed time, decision correctness, confidence before and after, opened evidence, overrides, and acceptance of unsupported advice. Blind outcome reviewers to condition where feasible. Account for participant and case clustering in the analysis rather than treating repeated decisions as independent observations.

Define timeout, withdrawal, infrastructure failure, and exclusion rules prospectively. Retain the assigned-case denominator and show missingness by condition, supplemented by sensitivity analyses rather than imputing invented answers. Establish retention periods and access controls for raw outputs and study records. Keep consent records and identifiable participant information outside the repository. Confidential material must not reach hosted providers without explicit approval for its data class. Tasks remain advisory and sandboxed, with severe errors recorded independently of average usability or speed.

4.7. Requirement validation and limitations

Table 4.2 returns to the implementation requirements in Chapter 3, Table 3.1. The F1–F4 identifiers denote functional capabilities; N1–N3 denote cross-cutting access safety,

auditability, and deployment flexibility. N1 makes the access/disclosure boundary explicit rather than treating governance alone as proof of runtime safety. For each requirement the table links an inspected mechanism and selected test evidence to an honest current status and pending acceptance step. The scenario and review instruments in Appendix C turn those gaps into testable questions. The source-level and test-level results concern RQ1–RQ3; their separation from effectiveness evidence addresses RQ4.

The principal limitations are small synthetic fixtures, pending expert judgments, and no measured model outputs or analyst observations. Article-level hits may miss relevant passages, while phrase checks can reward superficial compliance. Provider drift, corpus changes, policy differences, and learning effects would constrain later comparisons. The opt-in workflow and CI gates do not prove a deployment passed them. The executed suites establish bounded engineering behaviour; effectiveness, comparative model quality, and operational safety remain open.

Evaluation integrity is an additional research obligation, not evidence that any requirement has been empirically accepted: the current twelve-test report-gate rerun checks synthetic report consistency, while the application suites check selected implementation paths. Authentic expert reviews, a controlled live comparison, and approved observations of analyst decisions have not been supplied. The graph assertion store is not a graph-retrieval mechanism, and neither reranking nor graph benefit is inferred from these tests.

Table 4.2: Requirement ID to implementation/test evidence and current status, linked to Chapter 3, Table 3.1. Passing tests establish selected assertions only; empirical acceptance remains pending.

Requirement ID	Implementation and test evidence	Current status and next acceptance
F1 Governed knowledge	Membership, classification, approval and expiry gates; policy and lifecycle tests passed in the Java suite.	Engineering-checked, not operationally validated. Pending: multi-identity publication and revocation trials.
F2 Recoverable ingestion	Retained sources, leased jobs and fenced replacement; ingest/recovery tests include isolated PostgreSQL checks.	Engineering-checked. The screenshot upload is intercepted; failure-injection and sustained recovery trials remain pending.
F3 Relevant retrieval	Access-filtered vector–lexical search and article-level scoring; hybrid-search component tests passed.	Component-checked, effectiveness unmeasured. Ten relevance cases are developer fixtures; freeze independently judged rankings and compare baselines.
F4 Traceable answers	Revision-bound passages, citation binding and rechecked evidence viewer; citation/viewer tests passed and the browser capture was mocked.	Engineering-checked, entailment unvalidated. Independent claim–passage and temporal review remains pending.
N1 Access and disclosure safety	Policy-filtered retrieval, final evidence recheck and unavailable-state handling; authorization tests passed in isolation, while browser denial is mocked.	Partially checked, not an operational security guarantee. Pending: live-state identity boundaries, injection variants and disclosure traces.
N2 Reviewable operations	Redacted audit with protected stored rows and best-effort writes; unit and isolated database tests passed.	Mechanism checked, completeness unvalidated. Pending: event-loss monitoring and operational audit reconciliation.
N3 Deployment flexibility	Separate identity, storage and model configuration; configuration tests and frontend build passed.	Build/configuration checked, hosting unvalidated. Pending: deployment matrix, provider/data-flow audit and policy approval.

CHAPTER 5

Conclusion

5.1. Conclusions supported by the present work

This dissertation has developed an account of Sentinel KMS as a governed cybersecurity knowledge-management artefact. The design problem is not exhausted by connecting a language model to a vector index. An answer depends on which sources are available, which revisions are current, which passages support the response, and which user is permitted to inspect them. The inspected implementation gives those concerns explicit representation in document governance, retrieval, answer persistence, and evidence viewing.

The principal research question is answered at the level of implementation and bounded verification, not operational effectiveness. The committed system integrates explicit source and revision identities, database-backed access decisions, governed retrieval, and server-resolved evidence references. Later local work adds curated response playbooks, an incident-oriented interface, and evidence-dependent fallback behaviour, but those changes are not part of the inspected commit. Source inspection identifies the mechanisms; Java, frontend, browser, and report-validation checks establish only the behaviour exercised in their declared environments. None certifies a deployed system.

The subordinate questions have correspondingly bounded answers. RQ1 is addressed by the distinction between retained source or revision identity, derived retrieval text, and later evidence resolution under current permissions. RQ2 is addressed by hybrid retrieval and indicator lookups, authorised grounding, optional reranking, local abstention on missing grounding, and canonical article citations; the graph assertion store is not a retrieval branch. RQ3 is addressed by lifecycle and access checks, fenced indexing, repeated evidence resolution, and defensive prompt processing, with deployment security and injection resistance still requiring adversarial execution. RQ4 is addressed by the separation of deterministic evaluation mechanisms and executed report checks from the prospective provider, expert, and analyst evaluations in Chapter 4.

These answers also expose implementation trade-offs. An immutable reference can become unavailable under current access rules. A published revision can temporarily lack new embeddings. Authorised uploaded material can support grounding without having the article identity required by the article citation viewer. A passage supplied as context need not support every generated sentence. These are substantive characteristics of the implementation, not merely documentation details.

Whether the design improves analyst work remains unanswered. No controlled model comparison or participant study was conducted for this account. The passing engineering checks and scripted browser journeys do not measure retrieval quality, answer correctness, or analyst performance. The dissertation therefore does not conclude that Sentinel improves incident response, reduces analyst workload, or provides safe autonomous security operations.

5.2. Main contributions

The first contribution is the implementation account of knowledge governance integrated with durable ingestion and an advisory retrieval interface. Classification, workspace authority, approval, lifecycle state, and exact evidence identities are treated as part of knowledge access rather than as optional annotations appended to generated text. Source inspection identifies enforcement paths in the current code, while the evidence audit preserves their qualifications. The contribution lies in this integration and its analysis, not in proposing a new retrieval algorithm.

The second contribution is the explicit distinction between knowledge representations. Document content, immutable revisions, evidence snapshots, vector rows, graph assertions, and answer citations serve different purposes. In particular, the graph stores evidence-linked assertions but is not a graph-retrieval engine. This distinction prevents the presence of graph tables from becoming an unsupported claim of GraphRAG capability or multi-hop reasoning.

The third contribution is a layered evaluation procedure. It separates article retrieval, passage support, generated claims, citation links, abstention, temporal scope, access control, and analyst decisions. This is consistent with the component-level evaluation concerns raised by RAGAs and ARES, while adding the project-specific obligations created by governance and security-sensitive information.[5, 16] The procedure also preserves pending expert reviews and incomplete evidence instead of treating syntactically valid reports as completed scientific validation.

Supporting these contributions, the thesis package provides a traceable writing basis: the source-note exports, bibliographic catalogue, retrieved evidence, implementation audit, and offline test record accompany the LaTeX document. This supports subsequent correction and empirical extension. It does not replace the author's responsibility to inspect the cited work and verify the final interpretation.

5.3. Limitations

5.3.1. Literature and argument

Much of the literature is available here at abstract or handoff level. Such evidence supports descriptions of research directions and reported findings but leaves important questions about study design, sampling, leakage, baselines, and statistical uncertainty unresolved.

The synthesis is not an exhaustive systematic review. Unresolved bibliographic records are preserved transparently and do not support the substantive argument.

The literature also addresses different tasks: general knowledge sharing, CTI representation, question answering, malicious-code detection, and digital forensics. Their results cannot be pooled into a single predicted improvement for Sentinel. Positive and negative comparisons motivate controlled experiments rather than a presumption in favour of retrieval.[7, 17, 18]

5.3.2. Implementation and deployment

The description covers a committed implementation and distinguishes later local changes, neither of which is a certified deployment. The configured issuer, database permissions, outbound network rules, provider routing, and installed schema must be verified in each deployment. Local chat inference does not by itself establish local processing of embeddings, authentication, or storage.

The current graph has no retrieval or traversal integration. The ingestion pipeline's durable coordination does not guarantee that external work occurs exactly once. Evidence hashes and immutability controls preserve internal representations rather than guaranteeing source authenticity. Audit persistence is best-effort, and retained events therefore do not establish complete accountability. None of these limits invalidates the prototype, but all constrain what it can be claimed to demonstrate.

5.3.3. Evaluation and generalisation

The fixtures are small and synthetic, with pending expert review. Their phrases and identifiers encode intended behaviour rather than independently established ground truth. The stale-evidence case also exposes a substantive rubric question: whether a historical query should require abstention or a qualified historical answer. That decision must be resolved before the fixture is used to score correctness.

There are no measured provider-backed comparisons or analyst outcomes in this draft. Even after such measurements are collected, generalisation would be limited by the sampled users, documents, organisations, task definitions, and configurations. Severe errors must remain visible alongside average quality and speed; a favourable mean cannot establish that rare harmful advice is acceptable.

5.4. Future work and completion priorities

The first priority is evidence preparation rather than additional architectural complexity. The author should obtain and appraise the full texts most central to the argument, resolve remaining bibliographic records where possible, and review the corpus and case definitions with qualified cybersecurity reviewers. Review independence, qualifications, and adjudication must be documented outside the mere structure of an annotation file.

The next priority is an isolated, reproducible evaluation deployment. It should contain a frozen, authorised corpus and distinct test identities, with no dependency on production incidents or real confidential canaries. Access revocation, source replacement, stale vectors, hostile retrieved passages, and evidence-view failures should be exercised with recorded request traces. Provider-backed experiments should then compare declared search and generation conditions on identical eligible cases and retain missing outputs, timeouts, actual model identifiers, and costs where available.

Only after these foundations are established should the analyst study proceed. Institutional approval, informed consent, representative recruitment, and a prospective sample-size justification are prerequisites. The study should test whether evidence inspection and generated explanations change actual decisions, not just preferences for an interface. Confidence calibration and acceptance of unsupported advice should be retained as outcomes alongside correctness and elapsed time.

The existing optional reranker should first be assessed against its disabled condition with a declared candidate pool and configuration. Graph retrieval, semantic chunking, and model fine-tuning are possible later extensions, not missing prerequisites for describing the current artefact. Each should address a measured limitation. Introducing them simultaneously would make causal attribution difficult; the next architecture should be no more complex than the next defensible experiment requires.

5.5. Closing statement

The present work positions RAG as a testable knowledge-recovery component inside a broader governed system. Its strongest current contribution is the explicit treatment of authority, provenance, revision, and evaluation boundaries. Whether that design improves the work of cybersecurity analysts remains an empirical obligation. Keeping that distinction clear makes the dissertation a foundation for a credible study rather than a retrospective claim of success unsupported by results.

APPENDIX A

Implementation Evidence and Reproducibility Boundaries

A.1. How the evidence should be read

Chapter 3 attributes the implemented governance and evidence mechanisms to the inspected committed `main` revision, while identifying subsequent interface and prompt changes in the local working tree. Chapter 4 reports checks executed against that modified working tree. The return-to-requirements matrix in Table 4.2 uses F1–F4 for functional requirements and N1–N3 for non-functional requirements corresponding to Chapter 3, Table 3.1. In particular, N1 isolates runtime disclosure safety from F1's knowledge-governance workflow. A source definition establishes an available path, a passing isolated test establishes its specified assertion for selected inputs, and an intercepted browser response establishes presentation under a mock contract. None is an expert judgment of security advice or an end-to-end production demonstration.

A.2. Worked design scenario: publication, citation, and withdrawal

Consider a *proposed case*, not a recorded participant task: an editor submits a revised mitigation playbook; a different, authorised administrator reviews and publishes it. The current revision becomes eligible for asynchronous indexing. An analyst who may read the article asks a question, receives a passage-linked advisory answer, and opens the cited passage. Later the publication expires or the analyst's membership is revoked. The retained citation remains a historical record, while a subsequent passage request should become unavailable under the current policy. A trusted feed may follow a distinct initial-publication policy; it must not be mistaken for human approval in this scenario.

The case crosses several boundaries that a single screenshot cannot exercise. Publication state and revision identity must survive indexing delays; vector matches must be checked against the revision and the reader's rights; the model's prose must be reviewed for actual support; and passage viewing must recheck current eligibility. Table A.1 links these observations back to the named requirements rather than to a source-file inventory.

Table A.1: Design-scenario checkpoints and their evidential status; these are not outcomes of an executed end-to-end case.

Requirement ID	Mechanism in the proposed case	What would have to be observed
F1 Governed knowledge	Distinct proposer and approver; active membership, approved publication, and expiry controls.	Record allowed and denied transitions for separate identities, including later revocation; a mocked approval screen would not suffice.
F2 Recoverable ingestion	Retained upload bytes or source text, leased processing, and fenced index replacement.	Inject an interrupted worker and verify eventual eligible indexing without stale replacement or fabricated source content.
F3 Relevant retrieval	Policy-filtered lexical and vector candidates tied to current evidence.	Freeze the corpus and access policy; inspect ranked, authorised passages against independent relevance judgments.
F4 Traceable answers	Revision and passage identity carried into the answer and checked again by the evidence viewer.	Compare each material claim with the actual passage, then attempt the same citation after expiry or access loss.
N1 Access and disclosure safety	Read-policy checks at retrieval and response boundaries and safe unavailable views.	Use distinct real-state identities and adversarial content; inspect prompts, responses and logs for unauthorised disclosure.
N2 Reviewable operations	Redacted records of consequential publication and access actions, with best-effort writes.	Reconcile expected events with persisted events and disclose any audit-write losses rather than presuming completeness.
N3 Deployment flexibility	Independent identity, storage, and inference configuration.	Document the actual hosting topology and approved provider/data-flow settings for each trial.

A.3. Evidence identity and verification limits

An article citation conceptually carries the article identifier, revision identity and number, evidence-snapshot identity, and nonempty half-open passage offsets, with source version

and URI when available. The offsets refer to the retained evidence representation, which need not be identical to original upstream bytes. A historical reference does not confer continuing read access; current policy and publication state are rechecked at passage view. Nor does structural identity prove that the answer's claim follows from the passage.

The 29 September 2026 modified-tree verification comprised a twelve-test synthetic-report gate; 408 Java tests in 88 reports without failures, errors or skips; 91 frontend unit tests in 13 files; 17 Chromium tests; and a successful 133-module frontend build. Isolated PostgreSQL containers exercised selected backend paths, while at least the illustrative browser capture used intercepted authentication and application responses. The report validator had also passed an earlier audit, which is not added to the current run. The external implementation and evaluation audits hold exact source locations, while execution records and a source manifest identify commands, environments, hashes, and snapshot scope. These records support reproducibility of bounded engineering claims, not source authenticity, complete audit capture, model quality, or a human-participant dataset. The full-document build and citation checks are a further, separate editorial verification.

APPENDIX B

Source Evidence and Bibliographic Method

B.1. Research inputs and limits of inspection

The connected research notes and supplied reference list informed the literature map, but they are not a newly executed systematic search or a completed full-text quality appraisal. A companion reference catalogue preserves each supplied identifier, resolved bibliographic metadata, retrieval status, and evidence level. Unresolved records are not assigned invented metadata or used for substantive claims. The bibliography printed after these appendices contains the works actually cited in the dissertation; stable internal citation keys maintain consistency across chapters without determining printed reference numbers.

Table B.1 illustrates the distinction between a work's reported subject, the material inspected for this thesis, and a conclusion not warranted by that material. External source records are kept in the companion evidence material rather than reproduced as an internal file list here.

B.2. Claim-to-source examples

Table B.1: Illustrative claim-to-source boundaries in the literature argument.

Claim or use	Source and evidence available
RAG evaluation should distinguish retrieved context, answer faithfulness, and answer relevance.	RAGAs and ARES, with selected methods inspected directly; neither validates this project's outputs.[5, 16]
DSRM supplies six activities for artefact-oriented research.	Peffers et al., directly retrieved abstract; adoption of the framework does not complete each activity automatically.[14]
Some cyber tasks do not benefit from the tested RAG configuration.	PyPI detection, ForensicLLM, and RAGIntel abstracts; descriptions remain task-specific author-reported results.[2, 7, 17]

Claim or use	Source and evidence available
Graph construction can introduce noise and conflict.	Li et al., retrieved abstract, not a replicated quality benchmark.[10]
Knowledge sharing has organisational dimensions beyond retrieval.	Ipe and Yeboah, retrieved abstracts; transfer to Sentinel's users is an explicit design interpretation.[8, 20]
Retrieved text can carry indirect instructions.	Greshake et al., retrieved indexed abstract describing the attack surface; no complete defence claim follows.[6]

A deposited or indexed abstract is not always a usable abstract: bibliographic fragments can be mislabelled by an index. The catalogue therefore distinguishes the presence of an abstract field from an inspected substantive abstract. Where the text is deficient, the record must be downgraded or replaced before supporting a substantive claim. Direct inspection of selected full-text sections is likewise not equivalent to a complete risk-of-bias appraisal.

APPENDIX C

Proposed Review and Analyst-Study Instruments

These instruments are proposals for expert refinement. They are not responses collected from participants, validated scales, or evidence of ethics approval.

C.1. Scenario selection and requirement coverage

The existing answer dataset has nine synthetic cases, all with pending expert review. Table C.1 groups their intended challenges and identifies an additional lifecycle case that must be designed before an end-to-end trial. The first five rows describe developer-authored fixture categories, not observed answer performance. Expected phrases and forbidden identifiers cannot substitute for independent passage judgments or actual multi-identity access tests.

Table C.1: Reader-facing scenario prompts mapped to the requirement identifiers in Table 4.2; no expert or analyst outcomes are reported.

Scenario group	Requirement IDs	Prospective review question
Answerable and temporal disclosure (two cases)	F4	Does each claim follow from the right revision and disclose current versus historical status?
Unanswerable and ambiguous (two cases)	F3, F4	Is absence of authorised evidence distinguished from a question that requires clarification?
Conflicting and stale (two cases)	F1, F4	Are conflicting claims qualified, and is historical scope respected instead of mechanically forcing abstention?
Multi-document (one case)	F3, F4	Are multiple independent, readable passages needed and actually cited for the material claims?
Adversarial and access-denied (two cases)	N1, N2	Does untrusted text remain data and does a second identity fail to obtain protected content, including through traces?
Proposed lifecycle/recovery extension (not in seed dataset)	F1, F2, N1	After failed processing or revocation, is stale evidence neither served nor silently substituted?

C.2. Case review form

For each case, record the case identifier and scenario group, intended user role, workspace, frozen corpus and policy versions, question, requested temporal scope, answerability classification, and approved supporting passages. Each passage should carry its immutable identity and eligibility for that identity at the time of the task. Reviewers should specify both essential claims and unacceptable interpretations, including omissions that would make an otherwise true statement misleading. Record a different relevance set when a second identity has different permissions; do not reuse a globally visible answer key for a restricted subject.

The review should answer the following questions independently before adjudication:

1. Is the question sufficiently specified for the intended user and task?
2. Which authorised passages are relevant, and which claims do they actually support?
3. Is a current answer, a historical answer, a clarification, or an abstention appropriate?
4. Are there conflicts, superseded sources, or scope restrictions that must be disclosed?
5. What recommendation would constitute a severe error in this task?
6. Does the case depend on sensitive information that cannot be sent to a proposed provider?

Record reviewer identity, qualifications, review date, rationale, disagreements, and adjudication separately from the answer scores. The stale seed case currently demands abstention even though it asks about a superseded snapshot; reviewers must decide whether a qualified historical answer is instead legitimate before freezing the rubric. Model-generated labels may assist preparation but do not replace independent expert review.

C.3. Answer review form

For each baseline–case output, retain the actual model and retrieval configuration, eligible corpus and identity, output content digest, failures, and cost/latency observations when available. Segment material claims, associate their citations, and score support, citation entailment, completeness, temporal scope, abstention, and harmful advice. For an apparent disclosure, inspect retrieval, prompt, output, and evidence-view traces rather than the final answer alone. Include an "uncertain" judgement that can trigger adjudication instead of forcing an unsupported binary answer. Record missing output as missing output and preserve its reason; do not silently drop failed cases from a comparison.

The two independent reviews should be made without access to each other's labels. Record pre-adjudication agreement, then a separately identified adjudicator's resolution and rationale. A structurally complete review may still find that the answer was wrong or unsafe. The current fixture has no completed independent annotation.

C.4. Analyst task record

A proposed record contains a pseudonymous participant identifier, experience stratum, case and condition, allocation order, start and end timestamps, final decision, pre- and post-task confidence, evidence opened, overrides, and acceptance of unsupported advice. Outcome reviewers record decision correctness and error severity separately from perceived usefulness, blinded to condition where feasible. The assigned-case denominator includes timeout, withdrawal, and infrastructure failures with prespecified missingness categories. Actual participants, recruitment figures, consent dates, and responses must be supplied only after the approved study occurs.

Potential post-task prompts include: "Which evidence most influenced your decision?"; "Was any important information missing?"; and "Did you accept a recommendation you could not verify?" These prompts are exploratory, not a validated psychometric instrument. The final study should predefine scoring, counterbalanced allocation, missing-data handling, severe-error stopping criteria, retention, and access to participant records. The instruments here are templates for that approved future work, not a claim that the study or its ethics review has begun.

References

[1] Chon Abraham, France Bélanger, and Sally Daultrey. Promoting research on cyber threat intelligence sharing in ecosystems. *Journal of Cybersecurity*, 11(1), 2025. doi: 10.1093/cybsec/tyaf016. URL https://doi.org/10.1093/cybsec/tyaf016.

[2] Abeer Alhuzali. LLM-powered threat intelligence: a retrieval-augmented generation approach for cyber attack investigation. *PeerJ Computer Science*, 11:e3371, 2025. doi: 10.7717/peerj-cs.3371. URL https://doi.org/10.7717/peerj-cs.3371.

[3] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking Large Language Models in Retrieval-Augmented Generation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17754–17762, 2024. doi: 10.1609/aaai.v38i16.29728. URL https://doi.org/10.1609/aaai.v38i16.29728.

[4] Bhuwan Dhingra, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W. Cohen. Time-Aware Language Models as Temporal Knowledge Bases. *Transactions of the Association for Computational Linguistics*, 10: 257–273, 2022. doi: 10.1162/tacl_a_00459. URL https://doi.org/10.1162/tacl_a_00459.

[5] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.eacl-demo.16. URL https://doi.org/10.18653/v1/2024.eacl-demo.16.

[6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, pages 79–90. ACM, 2023. doi: 10.1145/3605764.3623985. URL https://doi.org/10.1145/3605764.3623985.

[7] Motunrayo Osatohanmen Ibiyo, Thinakone Louangdy, Phuong T. Nguyen, Claudio Di Sipio, and Davide Di Ruscio. Detecting Malicious Source Code in PyPI Packages with LLMs: Does RAG Come in Handy. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, pages 653–658. ACM, 2025. doi: 10.1145/3756681.3757042. URL https://doi.org/10.1145/3756681.3757042.

[8] Minu Ipe. Knowledge Sharing in Organizations: A Conceptual Framework. *Human Resource Development Review*, 2(4):337–359, 2003. doi: 10.1177/1534484303257985. URL https://doi.org/10.1177/1534484303257985.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

[10] Hongyi Li, Ze Shi, Chengwei Pan, Di Zhao, and Nan Sun. Cybersecurity knowledge graphs construction and quality assessment. *Complex & Intelligent Systems*, 10(1):1201–1217, 2024. doi: 10.1007/s40747-023-01205-1. URL https://doi.org/10.1007/s40747-023-01205-1.

[11] Fatma Yasmine Loumachi, Mohamed Chahine Ghanem, and Mohamed Amine Ferrag. Advancing Cyber Incident Timeline Analysis Through Retrieval-Augmented Generation and Large Language Models. *Computers*, 14(2):67, 2025. doi: 10.3390/computers14020067. URL https://doi.org/10.3390/computers14020067.

[12] Alex Nelson, Sanjay Rekhi, Murugiah Souppaya, and Karen Scarfone. Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile. Technical Report NIST SP 800-61r3, National Institute of Standards and Technology, April 2025. URL https://doi.org/10.6028/NIST.SP.800-61r3.

[13] Mohamad Amin Osman, Shahrul Azman Mohd Noah, and Saidah Saad. Ontology-Based Knowledge Management Tools for Knowledge Sharing in Organization—A Review. *IEEE Access*, 10:43267–43283, 2022. doi: 10.1109/ACCESS.2022.3163758. URL https://doi.org/10.1109/ACCESS.2022.3163758.

[14] Ken Peffers, Tuure Tuunanen, Marcus A. Rothenberger, and Samir Chatterjee. A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3):45–77, 2007. doi: 10.2753/MIS0742-1222240302. URL https://doi.org/10.2753/MIS0742-1222240302.

[15] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, November 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1410. URL https://aclanthology.org/D19-1410/.

[16] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 338–354. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.naacl-long.20. URL https://doi.org/10.18653/v1/2024.naacl-long.20.

[17] Binaya Sharma, James Ghawaly, Kyle McCleary, Andrew M. Webb, and Ibrahim Baggili. ForensicLLM: A local large language model for digital forensics. *Forensic Science International: Digital Investigation*, 52:301872, 2025. doi: 10.1016/j.fsidi.2025.301872. URL https://doi.org/10.1016/j.fsidi.2025.301872.

[18] Marco Simoni, Andrea Saracino, Vinod P, and Mauro Conti. MoRSE: Bridging the Gap in Cybersecurity Expertise with Retrieval Augmented Generation. In *Proceedings of the 40th ACM/SIGAPP Symposium on Applied Computing*, pages 1213–1222. ACM, 2025. doi: 10.1145/3672608.3707898. URL https://doi.org/10.1145/3672608.3707898.

[19] Takeshi Takahashi and Youki Kadobayashi. Reference Ontology for Cybersecurity Operational Information. *The Computer Journal*, 58(10):2297–2312, 2015. doi: 10.1093/comjnl/bxu101. URL https://doi.org/10.1093/comjnl/bxu101.

[20] Asiamah Yeboah. Knowledge sharing in organization: A systematic review. *Cogent Business & Management*, 10(1), 2023. doi: 10.1080/23311975.2023.2195027. URL https://doi.org/10.1080/23311975.2023.2195027.